Report-5: Python SDR Scripts for AERPAW

AERPAW Digital Twin, Experiment X1532

10 November 2026 (version 3)

Ismail Guvenc, Professor

Department of Electrical and Computer Engineering
North Carolina State University
Email: iguvenc@ncsu.edu

This report gives 14 Python scripts for I/Q collection, spectrum monitoring and link measurements on the virtual USRP radios of the AERPAW digital twin. For new users, it first explains I/Q samples, dBFS, SigMF and the path of the samples through the channel emulator. Then it gives a first-hour path through the scripts. The scripts use the UHD Python API and follow the rules of the virtual radios and the channel emulator. We tested each script in experiment X1532, together with srsRAN LTE, OCUDU 5G, OAI and the UHD2 sample. Seven new experiments give a recipe for labeled SigMF data sets and compare a spectrum monitor with the live metrics of an OCUDU cell. They also measure when channel-emulator commands act, test the other allowed transmit bands, check the late sample delivery, check the channel-emulator SNR with a bursty signal and measure the noise floor at several rates. The report also gives a catalog of public Python SDR repositories, which we did not run in the twin.

This report is generated by Claude Cowork and Claude Code. AI can make errors and the report is provided as is after light editing and revisions.

Contents

1. Introduction **3**

2. Background for Newcomers **4**

2.1 I/Q samples and the sample rate . . . 4
2.2 dBFS, clipping, SNR and EVM . . . 5
2.3 SigMF: records with metadata . . . 5
2.4 The AERPAW Python samples UHD1 and UHD2 . . . 5
2.5 The virtual USRP and the CHEM path . . . 7
2.6 How the scripts fit together . . . 8
2.7 Live metrics from the 5G stacks . . . 8

3. Using Claude with the AERPAW Digital Twin **10**

3.1 Getting started . . . 10
3.2 Your working folder . . . 10
3.3 Nodes of the experiment . . . 12
3.4 What Claude does in the twin . . . 12
3.5 Files that Claude made in our working folder . . . 13
3.6 Run a script of this report . . . 13
3.7 Sample commands . . . 14
3.8 Sample prompts . . . 15
3.9 Rules for work with Claude . . . 15

4. AERPAW Digital-Twin Rules for Python Radio Scripts **16**

5. The First Hour: a Path through the Scripts **19**

6. Tested Research Scripts **21**

6.1 Script 1: vusrp_io, V-USRP Rule Helper . . . 22
6.2 Script 2: iq_recorder, SigMF Recorder with CHEM Link Log . . . 23
6.3 Script 3: occupancy_monitor, Duty Cycle next to LTE and NR Cells . . . 24
6.4 Script 4: fixed_tune_scanner, Channelizer and Virtual Stepped Scan . . . 25
6.5 Script 5: channel_sounder, Zadoff-Chu Sounder and Power Delay Profile . . . 26
6.6 Script 6: siggen, Continuous Test-Signal Generator . . . 27
6.7 Script 7: meters, SNR, EVM and PAPR Meters . . . 28
6.8 Script 8: dl_capture, LTE and NR Downlink Capture and Decode . . . 29
6.9 Script 9: link_logger, CHEM, Ping and iperf3 in One CSV . . . 30
6.10 Script 10: doppler_cfo, Pilot-Tone CFO and Doppler Tracker . . . 31
6.11 Script 11: chem_sweeper, CHEM Sweep with Automatic Capture . . . 32
6.12 Script 12: chem_watchdog, CHEM Port Watchdog with reset_chem . . . 33
6.13 Script 13: preflight, V-USRP Self-Test . . . 34
6.14 Script 14: mat2sigmf, UHD2 Snapshots to SigMF . . . 35

7. New Twin Experiments (5 and 10 November 2026) **36**

7.1 R1: a labeled SigMF data set at several CHEM SNRs . . . 36
7.2 R2: a spectrum monitor next to a live OCUDU cell, with the gNB metrics . . . 37
7.3 R3: when CHEM frequency-offset and CIR commands act . . . 39
7.4 N1: the scripts in the other allowed TX bands . . . 41
7.5 N2: late sample delivery seen by the report scripts . . . 41
7.6 N4: the CHEM fixed SNR with a bursty signal . . . 42
7.7 N3: the noise floor against the sample rate on one node . . . 43

8. Public GitHub Repository Catalog **45**

8.1 UHD Python examples and textbook code . . . 45
8.2 Recorders and spectrum tools on the UHD Python API . . . 46
8.3 Channel sounders and link modems . . . 46
8.4 GNU Radio based AERPAW labs and recorders . . . 47
8.5 SigMF input and output, viewers and data sets . . . 47
8.6 Measurement DSP libraries . . . 48
8.7 Spectrum occupancy and energy detection . . . 48
8.8 Classification and AERPAW analysis code . . . 49
8.9 LTE and NR decoders . . . 49
8.10 SoapySDR, Pluto, RTL-SDR and other hardware . . . 50
9. Findings and Recommendations **52**
9.1 New findings from the script tests . . . 52
9.2 Limits of this report . . . 54
9.3 Recommendations . . . 54
10. Reproduce It **56**
10.1 Software versions . . . 56
10.2 Session frame . . . 56
10.3 Commands for the script tests . . . 57
10.4 Commands for the cell tests . . . 57
10.5 Commands for the new experiments of 5 November 2026 . . . 58
10.6 Commands for the new experiments of 10 November 2026 . . . 58
References **59**
Appendix A. Glossary **60**
Appendix B. Script Index and File Locations **63**
Appendix C. Partial and Failed Items **64**

1. Introduction

The Aerial Experimentation and Research Platform for Advanced Wireless (AERPAW) gives each experiment a development environment, the AERPAW digital twin. Each node of the twin has one virtual universal software radio peripheral (USRP) (V-USRP). A channel emulator (CHEM) connects the virtual radios of all nodes. Many researchers write their radio code in Python. Python code can control a USRP directly through the USRP hardware driver (UHD) Python application programming interface (API) [9] (`import uhd`), without GNU Radio. Python code also does most of the analysis of I/Q records: spectra, signal-to-noise ratio (SNR), error vector magnitude (EVM), occupancy, channel profiles and cell decoding.

Starting point. The AERPAW User Manual [8] gives two Python sample experiments: UHD1 for spectrum monitoring and UHD2 for I/Q collection. In our earlier tests in the twin, UHD1 stopped the channel emulator for all users, and UHD2 operated only after changes. A sweep, a high rate or a finite burst does not operate on the virtual radio. One long continuous stream at the frequency of the transmitter operates well. Section 2 explains why.

Scope of this report. We wrote 14 small Python scripts that follow the rules of the twin. They replace and extend the two samples (Table 1). They also give a test-signal generator, a channel sounder, SNR and EVM meters and a cell capture with a new radio (NR) decoder. The control scripts are a link logger, a carrier frequency offset (CFO) tracker, a CHEM sweeper, a CHEM watchdog and a self-test. We tested all 14 scripts in experiment X1532 on 4 November 2026. On 5 November 2026, we added three small experiments, and on 10 November 2026 four more (Section 7).

Who should read what. If you are new to I/Q data, read Sections 2, 4 and 5, and then the entry of the script that you need. If you know software-defined radios (SDRs) but not the twin, start with Section 4. If you want to repeat a result, go to Section 10.

The report has these parts. Section 2 gives the background, and Section 3 the work with Claude. Section 4 gives the rules of the twin, and Section 5 the first path through the scripts. Section 6 gives one entry for each script, and Section 7 the new experiments. Section 8 is the catalog of public repositories. Section 9 gives the findings, the limits and the recommendations, and Section 10 the versions and commands. The appendices give the glossary, the file locations and the open items.

This report is the fifth report of a series on the same twin experiment (X1532) [1, 2, 3, 4, 5, 6]. Report-1 gives GNU Radio experiments from GitHub with toy AERPAW deployments. Report-2 gives OCUDU 5G in the twin. Its Part-2 gives OCUDU unmanned aerial vehicle (UAV) flights and link drops. Report-3 gives orthogonal time frequency space (OTFS) and orthogonal frequency-division multiplexing (OFDM). Report-4 gives OAI 5G in the twin. Report-6 gives research concepts for the AERPAW Open Research on Autonomy (AORA) challenge. The title of each section links back to the table of contents.

2. Background for Newcomers

This section explains the ideas that the scripts use. Read it first if you have not worked with I/Q samples, with a software radio or with the AERPAW digital twin. Section 3 then tells how to work with Claude. Section 4 gives the rules of the twin, and Section 5 gives a first path through the scripts.

2.1. I/Q samples and the sample rate

A software-defined radio (SDR) mixes the radio signal down with a local oscillator (LO) at the center frequency f_c. The result is a stream of complex numbers. The real part is the in-phase (I) part, and the imaginary part is the quadrature (Q) part. Each complex sample gives the amplitude and the phase of the signal at one time. In Python, a record is a numpy array of type `complex64`. The UHD name of this format is `fc32` (two 32-bit floats).

The sample rate f_s sets the width of the spectrum in the record. A record at f_c and f_s covers $f_c - f_s/2$ to $f_c + f_s/2$. Fig. 1(a) shows the example of this report: $f_c = 3400$ MHz and $f_s = 2$ MS/s (mega-samples per second) cover 3399 to 3401 MHz. A tone at +100 kHz in the record is a radio signal at 3400.1 MHz. One second at 2 MS/s is 2 million samples, or 16 MB as `fc32`. A 30-s record is therefore 480 MB. The scripts of this report use 2 MS/s for link tests and 11.52 or 15.36 MS/s for long-term evolution (LTE) and 5G NR cell records.

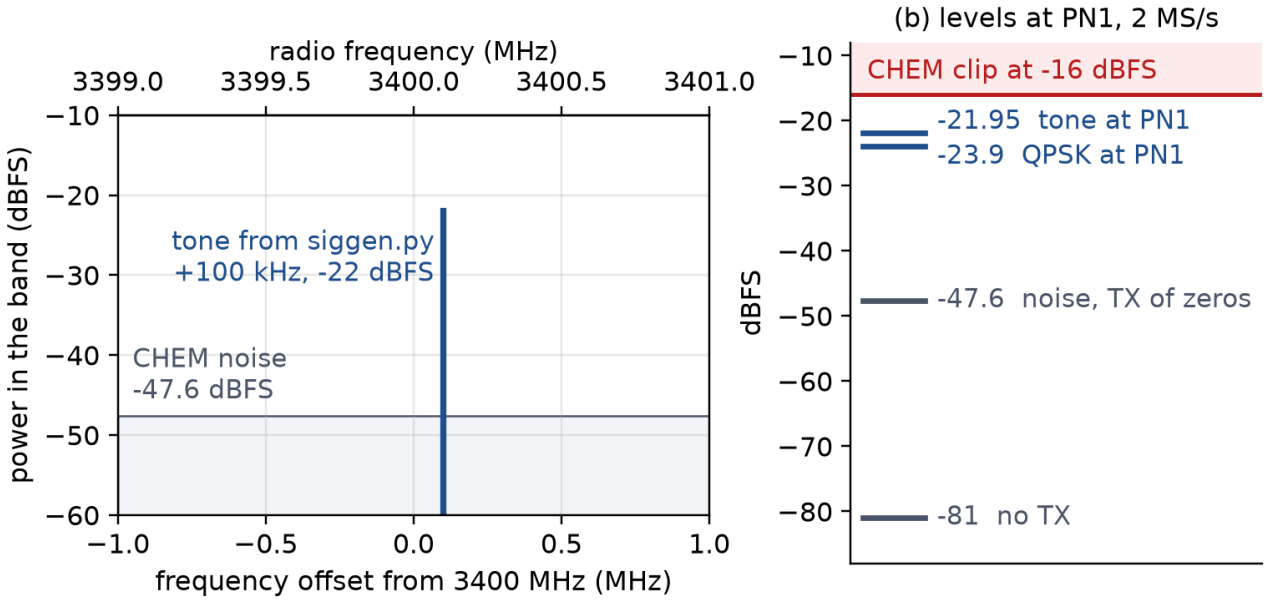


Figure 1: (a) One record at 3400 MHz and 2 MS/s covers 2 MHz of spectrum. The tone of `siggen.py` is at +100 kHz. The levels are total powers in the record band, not densities. (b) Measured levels at PN1 at 2 MS/s (Scripts 1 and 7): the CHEM clip, two received signals, the CHEM noise with a registered transmitter and the level with no transmitter.

2.2. dBFS, clipping, SNR and EVM

The scripts give power levels in dB relative to full scale (dBFS). A sample with $|x| = 1$ is at full scale. The level of a record is $P = 10 \log_{10}(\overline{|x|^2})$ dBFS. A signal at -20 dBFS has 1 % of the full-scale power. In the twin, the channel emulator (CHEM) clips the received level at -16 dBFS. Fig. 1(b) shows the levels that we measured. Keep each received signal below -18 dBFS, so that its peaks do not reach the clip.

The signal-to-noise ratio (SNR) is the signal power divided by the noise power. CHEM sets the SNR of each link over the full sample band. The error vector magnitude (EVM) compares the received symbols with the sent symbols. It is the root-mean-square (rms) error divided by the rms symbol, in percent. After a matched filter, the SNR from the EVM is approximately $-20 \log_{10}(\text{EVM})$. This SNR is per symbol after the matched filter. It is higher than the CHEM SNR over the full sample band by $10 \log_{10}(f_s/R_s)$, where R_s is the symbol rate. At 4 samples per symbol, the difference is 6.02 dB (Script 7). The peak-to-average power ratio (PAPR) tells how far the peaks of a signal are above its mean power. Its complementary cumulative distribution function (CCDF) gives the share of samples above each level. Script 7 (`meters.py`) calculates all these values.

2.3. SigMF: records with metadata

The Signal Metadata Format (SigMF) stores a record as two files. The file `.sigmf-data` holds the raw samples. The file `.sigmf-meta` holds JSON text with three parts:

- `global`: data type (`cf32_le`), sample rate, description, hardware and extensions.
- `captures`: the first sample, the center frequency and the start time of each capture segment.
- `annotations`: labels for sample ranges, for example the CHEM SNR of each second.

A namespace adds fields of its own. Our namespace `aerpaw:` stores the node name, the receive gain, the record statistics and the CHEM link state. This shortened example comes from the record of Script 2:

```
"global": {"core:datatype": "cf32_le", "core:sample_rate": 2000000.0, "aerpaw:link": "M1>M3",
           "aerpaw:stats": {"rms_dbfs": -21.95, "overflows": 0, ...}}
"captures": [{"core:sample_start": 0, "core:frequency": 3400000000.0,
              "core:datetime": "2026-11-04T21:56:53.826819Z"}]
"annotations": [{"core:sample_start": 0, "core:sample_count": 2000000, "core:label": "chem",
                 "aerpaw:chem_snr": 25.32, "aerpaw:chem_distance": 36.742,
                 "aerpaw:chem_freqOffsetHz": 0.0}]
```

The library sigmf-python reads and checks these files. The viewers IQEngine and inspectrum show them as spectrograms (Section 8). Because the CHEM state is in each file, each record is a labeled data set for machine learning (ML).

2.4. The AERPAW Python samples UHD1 and UHD2

The AERPAW User Manual gives two Python sample experiments on the UHD Python API. Most new users start from them.

- **UHD1, spectrum monitoring.** It tunes the LO to 231 frequencies from 100 MHz to 6 GHz. At each LO, it records 500 000 samples at 30.72 MS/s and writes a Welch power spectral density (PSD) to a `.mat` file.

- **UHD2, I/Q collection** (`get_IQ.py`). It records I/Q snapshots at 2 MS/s on the LOs 3500 and 3510 MHz for 0.5 s and writes each snapshot to a `.mat` file.

In our earlier tests in the twin, UHD1 stopped CHEM for all users 25 s after it started. UHD2 operated only after we moved its LOs to 3400 and 3410 MHz and set a run time of 30 s. It then received a median level of −32.8 dBFS at 3400 MHz (earlier test of 31 October, another TX signal). Section 4 explains why. Table 1 gives the scripts of this report that replace the two samples in the twin.

Table 1: The AERPAW Python samples and their replacements in the twin.

Sample	Problem in the twin	Replacement in this report
UHD1 spectrum monitoring	LO sweep, 30.72 MS/s and finite bursts. It stopped CHEM after 25 s.	`occupancy_monitor.py` and `fixed_tune_scanner.py`: one fixed tune at the TX frequency, channels cut out in software (Scripts 3 and 4)
UHD2 I/Q collection	Default LOs 3500 and 3510 MHz have no signal. Short run time. `.mat` output without metadata.	`iq_recorder.py`: one continuous SigMF record with the CHEM link log (Script 2). `mat2sigmf.py` converts the UHD2 output (Script 14).

2.5. The virtual USRP and the CHEM path

Fig. 2 shows the path of the samples in the twin. The twin has four nodes. FN1 and FN2 are the fixed nodes (LW1 and LW2), and PN1 and PN2 are the portable (drone) nodes. Older script options and data file names use the previous node names M1, M2, M3 and M4 for FN1, FN2, PN1 and PN2. For example, `--link 'M1>M3'` is the link from FN1 to PN1. Each node has one virtual USRP (V-USRP) of the B205 type. A Python script uses the V-USRP through the normal UHD Python API (`import uhd`). The V-USRP does not have a radio front end. It sends the samples of each transmit (TX) stream through the network to CHEM. CHEM keeps one link for each pair of a transmitter and a receiver. For each link, it applies path loss from the distance and the channel model, multipath taps, one Doppler shift for the link (no Doppler spread, Report-3), frequency offset and noise. Then it clips the level and sends the samples to the receive (RX) stream. CHEM connects only radios with the same center frequency and sample rate.

A program controls CHEM through transmission control protocol (TCP) port 5000, and the V-USRP gets its configuration through port 6000. We use our client `chemctl.py` and the CHEM client in `vusrp_io.py`. CHEM is shared by all users of the twin. A script that stops CHEM stops the experiments of the other users. Section 4 gives the rules that follow from this path.

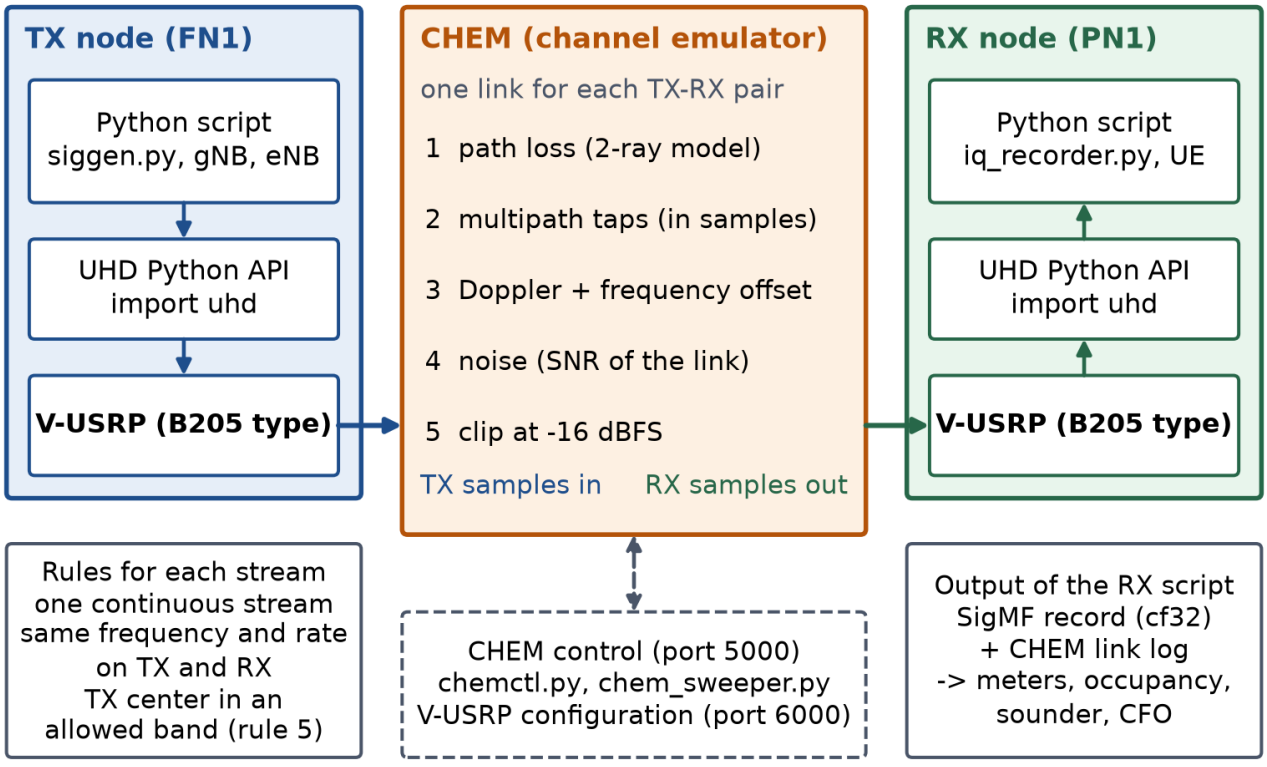


Figure 2: Signal path in the AERPAW digital twin. A TX script sends samples through the UHD Python API to the V-USRP. CHEM applies the channel of the link and sends the samples to the RX script. The CHEM control port and the V-USRP configuration port are on the CHEM server.

2.6. How the scripts fit together

Fig. 3 shows the 14 scripts in four groups. The sources on the TX node are `siggen.py`, the sounder and the cell stacks. The cell stacks are the srsRAN LTE evolved Node B (eNB), the OCUDU 5G next-generation Node B (gNB) and the user equipment (UE) of OAI or srsRAN. The capture scripts on the RX node write SigMF records. The analysis scripts read these records. They can run on the node or on any computer. The control scripts change CHEM, log the link or check the node. The helper `vusrp_io.py` (Script 1) is under all radio scripts.

The cell stacks couple to the scripts in three ways. First, a third node (PN2) records the cell with `dl_capture.py` or `occupancy_monitor.py`. Second, `link_logger.py` measures the Internet protocol (IP) link through the cell with ping and iperf3. Third, the 5G stacks give their own metrics (Section 2.7). Reports 2 and 4 give the OCUDU and OAI cells in detail.

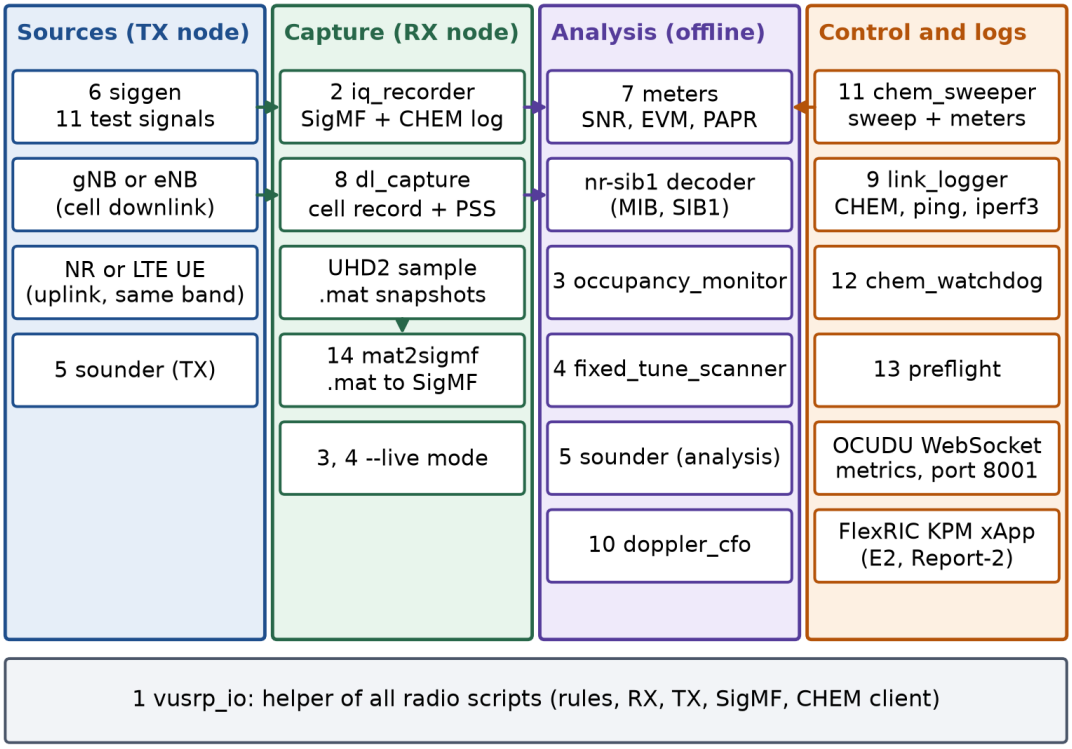


Figure 3: The scripts of this report and how they connect. The numbers are the script numbers. Arrows show the main data flow. All scripts read and write SigMF, so any capture script can feed any analysis script.

2.7. Live metrics from the 5G stacks

Report-2 shows two interfaces that give the state of the OCUDU gNB during operation. A Python script can use both.

- **OCUDU WebSocket interface (port 8001).** The section `remote_control` of the gNB YAML file turns on a WebSocket server on 127.0.0.1, port 8001. A client sends `{"cmd": "metrics_subscribe"}`. Then the gNB sends one JSON message each second. For each UE, a message gives the downlink (DL) and uplink (UL) modulation and coding scheme (MCS) and the bit rates. It also gives the HARQ (hybrid automatic repeat request) counters and the physical uplink shared channel (PUSCH) SNR. For the cell, it gives the used physical resource blocks (PRBs) for each slot of the time-division duplex (TDD) pattern. Our client `ws_tool.py` uses only the

Python standard library. The same interface also accepts commands, for example a change of the PRB quota.

- **FlexRIC key performance measurement (KPM) (E2).** A near-real-time radio access network (RAN) intelligent controller (near-RT RIC) gets key performance measurements (KPM) from the gNB through the E2 interface each second. Examples are the DL throughput (`DRB.UEThpDl`) and the used DL PRBs (`RRU.PrbUsedDl`). In the twin, the FlexRIC Python software development kit needs a newer SWIG than the nodes have. Therefore, the xApp is in C. A Python script reads the text log of the xApp.

With these metrics, a Python script can label an I/Q record with the state of the cell. `occupancy_monitor.py` can compare its duty cycle with the PRB use of the gNB (Section 7.2). `link_logger.py` can add the MCS and the bit rate to its CSV. `chem_sweeper.py` can use the gNB SNR to confirm that a CHEM change has acted. The interfaces are specific to OCUDU (WebSocket) or need an E2 agent (KPM). The srsRAN LTE eNB and the OAI UE of this report do not give them.

3. Using Claude with the AERPAW Digital Twin

Claude Code is an artificial intelligence (AI) agent that operates in a terminal, for example Windows PowerShell. It reads and writes files, runs commands, and connects to the nodes of an AERPAW experiment through secure shell (SSH). Claude Cowork gives the same agent in the Claude desktop application. We used both to write and test the Python scripts of this report, and to write the report. You give Claude a task in plain English. Claude then writes the scripts, runs them on the nodes, reads the logs and gives you the numbers.

3.1. Getting started

Fig. 4 shows how to start. The left column gives your steps in the AERPAW experiment portal:

1. Define the fixed and portable nodes of the experiment.
2. Save the experiment.
3. Start the development session. After this step, you cannot change the nodes.
4. Wait for the e-mail that tells you that the virtual environment is ready.
5. Download the OpenVPN profile (`.ovpn`) and the Manifest, which lists the IP address of each node.

Then you give these two files to Claude. The right column gives the steps of Claude. Claude reads the Manifest (A) and helps you to connect the virtual private network (VPN) (B). Then it tests SSH to each node (C) and checks the OEO (the AERPAW operator interface) console and the channel emulator (D). Claude connects or changes a VPN only when you ask it to do so. After step D, you can run experiments.

3.2. Your working folder

You can use any folder on your computer. Do these steps:

1. Make a new folder, for example `aerpaw_work`, in a location that you select.
2. Open Windows PowerShell and go to that folder, for example `cd aerpaw_work`.
3. Type `claude` to start Claude Code in that folder.

Claude then writes all its files in this folder: scripts, logs, results, figures and reports. In this report, all folder names are relative to this working folder. Folder names such as `node_tools` and `gnuradio_deploy` are examples from our working folder. Claude can use other names in your folder.

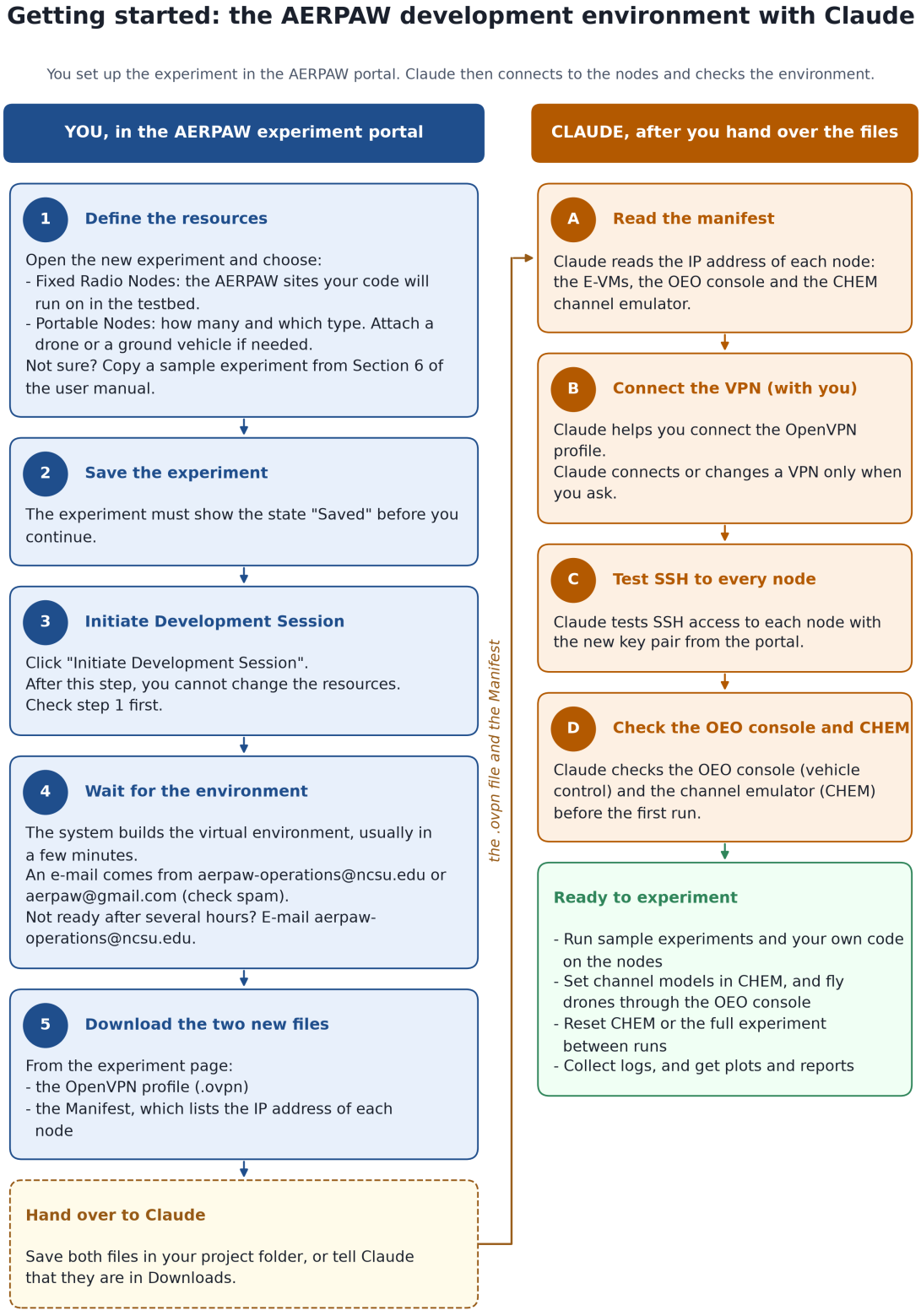


Figure 4: Getting started with Claude in the AERPAW development environment. You do the portal steps 1 to 5. Then Claude does the steps A to D.

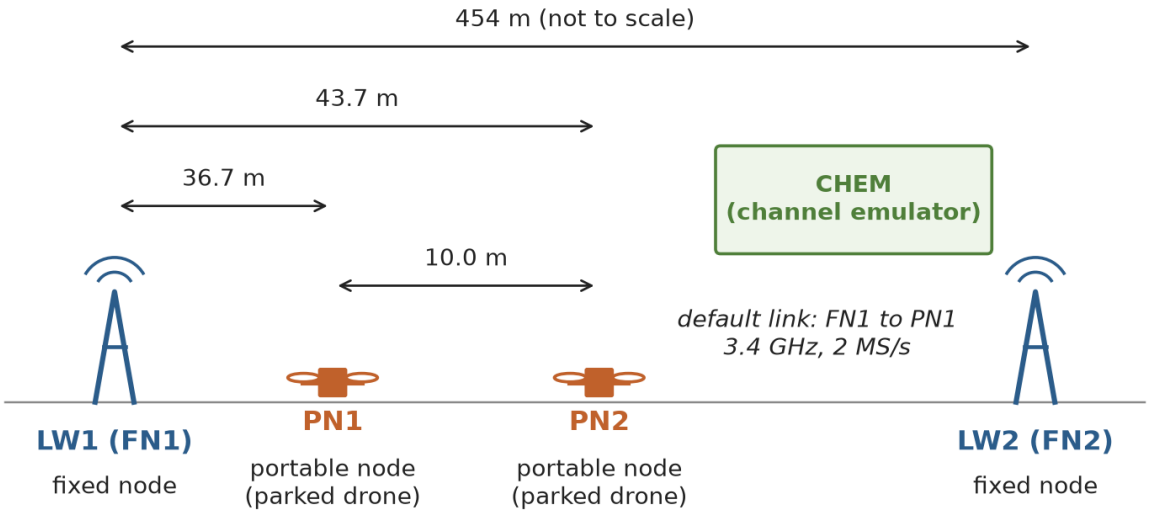


Figure 5: Nodes of experiment X1532: two fixed nodes and two portable nodes (drones on the ground). The distances are the CHEM link distances. The drawing is not to scale. The label "default link" gives the default link of this report.

3.3. Nodes of the experiment

This report uses one AERPAW experiment (X1532) with two fixed nodes and two portable nodes (Fig. 5). The fixed nodes are LW1 (FN1) and LW2 (FN2), 454 m apart. The portable nodes PN1 and PN2 are drones. They stay on the ground, 36.7 m and 43.7 m from LW1. Each node has one virtual USRP (V-USRP), and the channel emulator (CHEM) connects all the virtual radios. Most tests of this report use the link from FN1 to PN1. The cell tests use PN2 as a monitor node.

You can reserve more or fewer fixed and portable nodes in the AERPAW experiment portal. The scripts of this report also operate with other node pairs. Change only the link name (for example `--link 'M1>M3'`) and the node addresses, or tell Claude which nodes to use.

3.4. What Claude does in the twin

For the scripts of this report, Claude did these tasks without manual commands:

- Write a Python script on the UHD Python API (`import uhd`) and copy it to the nodes (`/root/claude_tools/aerpaw_py/`).
- Apply the rules of Section 4 in the code. For example, use one continuous stream, the same frequency and rate on TX and RX, and a TX in an allowed band (we used 3.4 GHz).
- Reset the experiment (`/root/reset.sh`) and set the CHEM channel model before each radio session.
- Start the srsRAN LTE, OCUDU 5G or OAI stacks when a script needs a cell, and start iperf3 and ping.

- Start the transmitter and the receivers on the nodes FN1, FN2, PN1 and PN2, stop them, and copy the logs to your working folder.
- Calculate the metrics (SNR, EVM, duty cycle, power delay profile (PDP), CFO), make the figures, and write the result paragraph.
- Run several tasks at the same time with subagents. A radio lock on FN1 lets only one radio test operate at a time.

3.5. Files that Claude made in our working folder

The table gives the files that Claude made for the scripts of this report. The names are relative to our working folder:

Table 2: Files that Claude made for this report.

File or folder	Function
`node_tools/aerpaw_py/*.py` (on the nodes: `/root/claude_tools/aerpaw_py/`)	The 14 scripts of Section 6.
`gnuradio_deploy/results/pyb_01` to `pyb_14`	Scenario, setup, metrics, paragraph and figure of each script test (`result.json`, `fig.png`). The folder `logs/` has the logs.
`node_tools/chemctl.py`, `chem_logger.py`, `iqcap.py`	CHEM control, CHEM link log and a GNU Radio IQ capture (Report-1).
`reports/`, `research_notes/`	The research report and the notes on public Python SDR repositories (Section 8).
`gnuradio_deploy/results/pyb_15` to `pyb_21`	The new experiments of 5 and 10 November 2026 (Section 7).
`gnuradio_deploy/work/deployX/sess*.sh`	The session scripts: radio lock, reset, TX and RX commands, log copy (Section 10).
`report_python/`	This report: `main.tex`, the section files, `figures/` and `tools/` (figure scripts).

3.6. Run a script of this report

You do not have to give file names to Claude. Do these steps:

1. Drag this report (the PDF file) into the Claude window.
2. Tell Claude in plain words what to do. For example: “Run Script 5 of this report again with the channel impulse response (CIR) tap set dense8 and compare the PDP with the report.”

Claude reads the script entry and its result folder. Then it runs the same scenario on the nodes, or a changed scenario, and compares the numbers with the report. If the files of an earlier run are in your working folder, Claude also uses them.

Fig. 6 shows the form of such a request for Script 11. The time-stamped lines and the JSON rows are shortened lines of our session log of 4 November 2026. The answer gives the values of Script 11.

Claude Code | your working folder

```
> Run Script 11 with QPSK from 30 dB to 0 dB and compare the
  measured SNR with the CHEM value.

* Read the report: Script 11, FN1 sends QPSK, sweeper on PN1
* Bash  sess.sh S2  (radio lock on FN1, reset, sweep)
  [16:54:34] radio lock taken (S2)
  [16:55:39] experiment reset, CHEM back after 60 s
  [16:55:39] CHEM> set 2_RAY -0.3 --shadow 0
  [16:57:41] --- SNR sweep: QPSK TX, CHEM FIXED_SNR 30..0 dB
  {"value": "30", "chem_snr": 30.0, "snr_da_db": 30.0, "evm_pct": 1.59}
  {"value": "10", "chem_snr": 10.0, "snr_da_db": 10.01, "evm_pct": 15.75}
  {"value": "0",  "chem_snr": 0.0,  "snr_da_db": 0.01, "evm_pct": 49.78}

* Answer:
  CHEM SNR          Measured SNR                  EVM
  30 dB             30.00 dB                      1.59 %
  10 dB             10.01 dB                      15.75 %
  0 dB              0.01 dB                       49.78 %
  Error             0.01 dB or less at all 8 points
```

Figure 6: A request to Claude Code for Script 11, the logged steps of the session and the answer. The request is an example. The log lines are shortened lines of our session log (4 November 2026, control-computer clock). The figure is drawn again in a light theme.

3.7. Sample commands

These commands operate on a node through SSH. Claude uses them in its scripts, and you can also type them. `chemctl.py` is our own client for the CHEM interface. It is not in the AERPAW image. Copy it to the node with the scripts.

```
# connect to FN1 (LW1); FN2, PN1 and PN2 use other addresses of the same form
ssh -i <your_key> root@192.168.XXX.YYY
# on the node: load the experiment environment (necessary for each radio program)
source /root/.ap-set-experiment-env.sh; source /root/.bashrc
# reset the experiment (all C-VMs and CHEM, approximately 55 s), then set the channel
/root/reset.sh
python3 /root/claude_tools/chemctl.py set 2_RAY -0.3 --shadow 0
python3 /root/claude_tools/chemctl.py links            # distance, path loss and SNR of each link
# self-test, then a tone on FN1 and a 30 s SigMF record on PN1 (3.4 GHz, 2 MS/s)
python3 /root/claude_tools/aerpaw_py/preflight.py check
python3 /root/claude_tools/aerpaw_py/siggen.py tone --secs 60                      # on FN1
python3 /root/claude_tools/aerpaw_py/iq_recorder.py rec1 --secs 30 --link 'M1>M3'  # on PN1
```

3.8. Sample prompts

These requests use only the script numbers and the names of this report. They are examples. We did not run all of them:

- "Run Script 13 (preflight) on FN1 and PN1. Then run Script 11 with a fixed SNR from 30 dB to 0 dB and compare the SNR values with the report."
- "Run Script 3 next to the OCUDU cell with FN2 as the monitor node. Log the gNB WebSocket metrics and compare the duty cycle with the PRB use."
- "Fly PN1 on `lw1-to-lw2.plan` and run Script 10 on PN1 during the flight. Compare the measured Doppler with the CHEM log."
- "Use Script 2 to record 60 s of the srsRAN LTE downlink at 11.52 MS/s, and keep the RX level below the clip level."
- "Deploy the GitHub repository <URL> with the rules of Section 4, and write an entry in the form of this report."

3.9. Rules for work with Claude

- Claude connects, starts or changes a VPN only when you ask it to do so.
- Tell Claude to reset the experiment between runs. A reset restarts all C-VMs and CHEM, and it takes approximately 1 min.
- If CHEM stops, tell Claude to run `reset_chem` on FN1 (Script 12). You do not need to wait for the AERPAW operators.
- If several agents use the radios, tell them to use the radio lock on FN1. Only one radio test can operate at a time.
- Tell Claude to time-stamp events on the nodes. In our sessions, the clock of the control computer was 13.3 s behind the node clocks (Section 7.3).
- Read the logs and the numbers before you use a result. Claude can make errors.

4. AERPAW Digital-Twin Rules for Python Radio Scripts

The V-USRP is not a real USRP. It sends the samples of each TX stream to the CHEM server. CHEM applies the channel of each link (path loss, noise, frequency offset, Doppler and multipath taps) and sends the samples to each RX stream. CHEM is shared by all users of the twin. Therefore, a script that stops CHEM stops the experiments of other users, as UHD1 did. Table 3 gives the rules that a Python radio script must follow. The rules come from our earlier CHEM tests in experiment X1532 (findings L3 to L28 of our debug report [7]). Some come from the tests of this report. The column "In the code" tells how the helper module `vusrp_io.py` (Script 1) applies the rule. The values are from our sessions. They can change with a different CHEM version.

Table 3: Rules for Python radio scripts on the V-USRP and CHEM of the AERPAW digital twin.

Rule	Context and evidence	In the code (`vusrp_io.py`)
1. Continuous TX stream	A bursty TX (`tx_sob`/`tx_eob` in each packet) does not reach the RX. Put zeros between the bursts.	One start of burst, zero fill, one end of burst at the end. `siggen.py` repeats one period.
2. Continuous RX stream	A finite RX burst (`num_done`) returns 0 samples and no error. `start_cont` and the UHD 4.9 function `recv_num_samps()` operate.	Refuses `num_done`. Uses `start_cont`, a read loop and `stop_cont`.
3. Same center frequency and rate on TX and RX	CHEM connects only radios with equal center frequency and rate. An RX retune of 500 kHz gives noise only. Do not retune during a stream. Put a channel offset in the baseband.	Sets rate, frequency and gain once before the stream. Refuses a retune during a stream.
4. Rate 1 to 23.04 MS/s	At 96 to 200 kS/s, no signal arrived. At 250 to 500 kS/s, the stream had gaps (debug report, L12). At 30.72 MS/s, there is no link. Use 1 to 3 MS/s for link tests and up to 15.36 MS/s for cell records.	Refuses 0.5 and 30.72 MS/s (Script 1). All records at 2, 11.52 and 15.36 MS/s had no overflow.
5. TX center in an allowed band	The V-USRP checks only the TX center frequency [3]. It accepts 6.765–6.795, 13.553–13.567, 26.957–27.283, 40.66–40.70, 433.05–434.79, 860–880, 902–928 and 3300–3450 MHz. It refuses other frequencies, for example 3500 MHz. A link at 914 MHz operated. Keep the full sample band inside the range.	Refuses a TX center outside these ranges. Warns when $f_c \pm f_s/2$ is outside the range.
6. RX level below −16 dBFS	CHEM clips the RX level at −16 dBFS. In the LTE load test, 35 % of the samples at PN2 were at the clip (Script 3).	Warns above −18 dBFS. `meters.py` gives the clip fraction.
7. CHEM noise only with a registered TX	CHEM adds the link noise only when a TX is registered. At PN1, the level was −81.0 dBFS with no TX and −47.6 dBFS with a TX of zeros (Script 1).	`siggen.py zeros` registers a TX for a noise record.
8. CIR taps in samples	CHEM applies the multipath taps in samples, not in seconds. At 2 MS/s, one tap is 0.5 μs (Script 5).	`channel_sounder.py` gives the delays in samples and in μs.

Rule	Context and evidence	In the code (`vusrp_io.py`)
9. No propagation delay	CHEM applies the multipath taps, but it adds no delay for the distance. A timing advance or a time of flight does not change with the distance [3].	`channel_sounder.py` gives delays relative to the first tap only.
10. CHEM controls act only on links that exist	A fixed SNR, a frequency offset or a CIR acted within 0.55 s of the command (R1 and R3, node clocks). A command acts only on links that exist when it runs. A CIR set with no RX stream acted on no link (Script 5).	`chem_sweeper.py` keeps one RX stream open for the full sweep. It waits 3 s for each point (version 1: 16 and 9 s).
11. Drop the first second	CHEM shows a 0 m link for 0.25 to 0.75 s after a radio registers.	Drops 1 s at the start of each record.
12. Absolute gains and `sc16`	Normalized gains gave no usable link. The V-USRP does not accept `sc8`.	Absolute gains (TX 60, RX 40), `sc16` wire format.
13. Small send calls	A send of more than 400 kB fails ("Too many fragments").	Each TX send is at most 25 000 samples (200 kB as fc32). Each RX read is exactly `max_num_samps`.
14. Program start and stop	Source `/root/.ap-set-experiment-env.sh` before a radio program. The V-USRP threads do not stop and ignore SIGTERM. `/dev/shm` holds only 64 MB.	Checks the environment, writes records to disk and stops with `os._exit`.
15. Reset before each session	`/root/reset.sh` restarts all control virtual machines (C-VMs) and CHEM in approximately 55 s. The CHEM ports stopped for 12.0 s during the reset (Script 12).	`chem_watchdog.py` waits for the ports. Use a grace time above the reset time.
16. Restart a stopped CHEM	`reset_chem` on FN1 restarts CHEM. It took 15.9 s in our test (Script 12).	`chem_watchdog.py` runs `reset_chem` once after a stop.
17. One radio test at a time	A radio lock on FN1 lets only one agent use the radios. After a radio stops, wait 20 s before the next radio starts on that node.	`preflight.py` opens the device. Wait 20 s after it.
18. Node clocks for time stamps	On 5 November 2026, the clock of the control computer was 13.3 s behind the node clocks (R3). The node clocks agreed within 0.1 s.	The scripts time-stamp records and CHEM rows on the node.
19. Late sample delivery	The twin sometimes delivers RX samples 0.4 to 1.1 ms late, in steps, with a 2 Hz pattern on several nodes (debug study of 6 and 7 November 2026 [7]). The sample values are correct. A real-time receiver can miss its deadline.	`RX.read(..., lag_log=True)` records the arrival lag (N2).

Consequences for public code. A public script ports easily if it opens one continuous stream at a fixed frequency. It does not port if it sweeps, sends bursts or needs a GPS-disciplined oscillator (GPSDO). A retune loop (for example UHD1) receives a signal only on the LO that is equal to the TX frequency. Therefore, an occupancy study in the twin covers the channels inside one tuned band at the TX frequency. A study across bands needs the real testbed.

Common mistakes. These mistakes gave no signal or wrong numbers in our tests:

- A retune during a stream, a frequency sweep or a finite burst (`num_done`). The record has noise only or 0 samples, and no error appears.
- A TX center frequency outside the allowed bands of rule 5, for example 3500 MHz. The V-USRP refuses it.
- Different center frequencies or rates on TX and RX. CHEM does not connect the two radios.
- A noise record with no transmitter. CHEM adds noise only when a TX is registered, so the floor is 33 dB too low.
- An RX level above −18 dBFS. CHEM clips at −16 dBFS, and the power and SNR values are wrong.
- The RX starts before the TX, or a radio starts again within 20 s on the same node. CHEM can show a false 0-m link.
- Times from the control computer. In our sessions, its clock was 13.3 s behind the node clocks. Time-stamp on the nodes.

5. The First Hour: a Path through the Scripts

This section gives a first path through the scripts for a new user. It goes from a self-test to a labeled data set in six steps. Each step gives the command and the result that we measured, so that you can compare. The full entries of the scripts are in Section 6. The path uses FN1 as the TX node and PN1 as the RX node, as in most tests of this report. Use your own node pair if you want.

Step 0: prepare the nodes. Copy the folder `aerpaw_py` (the 14 scripts, `ds_manifest.py`, `r6_ana.py` and the folder `_vendor` with sigmf-python) to each node, for example to `/root/claude_tools/aerpaw_py/`. In each SSH shell, load the experiment environment before a radio program. Reset the experiment and set the CHEM channel model before the session. Hold the radio lock, if other people or agents use the same nodes (Section 3).

```
source /root/.ap-set-experiment-env.sh; cd /root/claude_tools/aerpaw_py
/root/reset.sh                                  # on FN1: all C-VMs and CHEM, about 55 s
python3 /root/claude_tools/chemctl.py set 2_RAY -0.3 --shadow 0
```

Step 1: self-test (Script 13). Run `preflight.py check` on FN1 and PN1. It checks the environment, the CHEM ports, the V-USRP and the rate rules. Then run a tone loop. *We measured:* 7 of 7 items passed, tone in the FFT bin at 100 006.1 Hz (the bin nearest to 100 kHz, bin width 30.5 Hz), 71.5 dB above the floor, level −21.95 dBFS, no overflow.

```
python3 preflight.py check                      # on FN1 and on PN1
python3 preflight.py tx --secs 40               # on FN1
python3 preflight.py rx --secs 5                # on PN1, 10 s later
```

Step 2: your first SigMF record (Scripts 6 and 2). Start a quadrature phase-shift keying (QPSK) signal on FN1, wait 10 s, and record 10 s on PN1. Wait 20 s after a radio program stops before the next one starts on the same node. *We measured:* level approximately −24 dBFS, no overflow, one annotation with the CHEM state for each second.

```
python3 siggen.py qpsk --rms-dbfs -20 --secs 120                    # on FN1
python3 siggen.py qpsk --rms-dbfs -20 --save qpsk_ref.c64           # on PN1: one period, no radio
python3 iq_recorder.py rec1 --secs 10 --link 'M1>M3' --label first  # on PN1
```

The same record in your own Python code, with the helper of Script 1:

```
import vusrp_io as v
rx = v.RX(2e6, 3.4e9, 40)               # rate, frequency and gain are fixed before the stream
x, t0 = rx.read(int(2e6 * 10), 1.0)     # 10 s after the first second (dropped)
rx.stop(); print(v.dbfs(x), "dBFS")
v.write_sigmf("rec2", x, 2e6, 3.4e9, "my first record", t0)
v.hard_exit(0)                           # the V-USRP threads do not stop by themselves
```

Step 3: measure the link (Script 7). *We measured* at the CHEM link budget (23.45 dB): data-aided SNR 23.45 dB, QPSK EVM 3.35 %.

```
python3 meters.py rec1 --ref qpsk_ref.c64 --mod qpsk --plot rec1_meters.png
```

Step 4: change the channel (Script 11). Let `chem_sweeper.py` set the CHEM SNR, record and measure each point. With `--save-iq`, each point becomes one labeled SigMF record. Section 7.1 gives the data-set recipe and shows that a settle time of 3 s is sufficient for a fixed SNR.

```
python3 chem_sweeper.py noise 30,20,10,0 --link 'M1>M3' --ref qpsk_ref.c64 --mod qpsk \
        --settle 3 --secs 2 --save-iq --restore LINK_BUDGET --out ds_qpsk
```

Step 5: watch the spectrum (Script 3). Record a TX of zeros (noise floor) and the four-channel bursts, and measure the duty cycle of each channel. *We measured* 1.0, 0.5, 0.3 and 0.0 for the set values 1, 0.5, 0.25 and 0.

```
python3 siggen.py zeros --secs 40                          # on FN1
python3 iq_recorder.py noise_zeros --secs 4                # on PN1
python3 siggen.py bursts --rms-dbfs -24 --secs 40          # on FN1, 20 s after the stop of zeros
python3 iq_recorder.py bursts --secs 5                     # on PN1
python3 occupancy_monitor.py bursts --noise noise_zeros --block-ms 0.5 --plot \
        --channels=-750e3:250e3,-250e3:250e3,250e3:250e3,750e3:250e3
```

Step 6: go further. Record a cell with `dl_capture.py` (Script 8). Log a 5G link with `link_logger.py` (Script 9). Compare a spectrum monitor with the gNB metrics (Section 7.2).

Section 4 lists the common mistakes that gave no signal or wrong numbers in our tests.

6. Tested Research Scripts

Table 4 gives the 14 scripts. All scripts are in `node_tools/aerpaw_py/` in our working folder and in `/root/claude_tools/aerpaw_py/` on the nodes. They need only Python 3.10, numpy, scipy, matplotlib and the UHD 4.9 Python API of the node image. The SigMF scripts use a private install of sigmf-python 1.14.0. Unless an entry tells otherwise, FN1 sent at 3400 MHz and 2 MS/s with TX gain 60, and PN1 received with RX gain 40. The CHEM model was 2-ray (−0.3, no shadowing), set after `/root/reset.sh`. PN1 was on the ground, 36.7 m from FN1. The numbers in the entries come from the result files without change.

Table 4: The 14 tested Python scripts (result folders `pyb_01` to `pyb_14`).

#	Script	Function	Couples with	Key result
1	`vusrp_io.py`	Rule helper for the UHD Python API	All scripts	0 overflows in 10 records
2	`iq_recorder.py`	SigMF recorder with CHEM link log	UHD2 (replaces it)	25 of 25 records valid SigMF
3	`occupancy_ monitor.py`	Duty cycle per channel	srsRAN LTE, OCUDU, OAI	LTE duty 0 to 0.2 idle, 1.0 with load
4	`fixed_tune_ scanner.py`	Digital channelizer, virtual scan	UHD1 (replaces sweep)	Alias at 10 ms revisit
5	`channel_sounder.py`	Zadoff-Chu sounder, PDP	CHEM CIR, Script 11	Taps to 0.01 dB
6	`siggen.py`	Continuous test signals	All RX scripts	7 kinds on air
7	`meters.py`	SNR, EVM, PAPR CCDF	CHEM fixed SNR	SNR to 0.01 dB
8	`dl_capture.py`	Cell DL capture, primary synchronization signal (PSS), NR decode	srsRAN LTE, OCUDU, nr-sib1	System information block 1 (SIB1) in 16 of 26 windows
9	`link_logger.py`	CHEM, ping and iperf3 in one CSV	OCUDU, Open5GS, OAI UE	4.59 Mbit/s TCP DL
10	`doppler_cfo.py`	Pilot-tone CFO tracker	CHEM fo, Script 2	0.001 Hz median error
11	`chem_sweeper.py`	CHEM sweep with capture	Scripts 5, 6, 7	8 points, 18.2 s each
12	`chem_watchdog.py`	CHEM port watchdog	`reset_chem`	Ports up 26.9 s after stop
13	`preflight.py`	Self-test before a campaign	All scripts	7 of 7 items passed
14	`mat2sigmf.py`	UHD2 `.mat` to SigMF	UHD2 sample	172 snapshots, 2 records

6.1. Script 1: vusrp_io, V-USRP Rule Helper

File: `node_tools/aerpaw_py/vusrp_io.py` **Results:** `gnuradio_deploy/results/pyb_01`
Status: Tested in the twin (experiment X1532). **Couples with:** all scripts of this report
Public start point: UHD 4.9 Python API (`uhd.usrp.MultiUSRP`) and its function `recv_num_samps()`.

Purpose. `vusrp_io.py` is the shared helper module of the other 13 scripts. It wraps the UHD Python API and stops a script before the script breaks a V-USRP rule (Table 3). It refuses a rate outside 1 to 23.04 MS/s, a TX center outside the allowed bands of rule 5, a finite RX burst and a retune during a stream. It drops the first second of each record and warns above −18 dBFS. The module also has a SigMF writer and reader, a raw-file reader, a CHEM client and a CHEM poller (`ChemPoller`).

Coupling. Each script of this report imports the module. A public UHD script can also import it. Then the classes `RX` and `TX` replace the finite `num_done` burst with a continuous stream.

Command line.

```
python3 vusrp_io.py      # rule self-test without radio
# in a script:  import vusrp_io as v;  rx = v.RX(2e6, 3.4e9, 40);  x, t0 = rx.read(int(2e6*30), 1.0)
```

Results. The self-test refused 0.5 MS/s, 30.72 MS/s and a TX at 3500 MHz. It accepted 2 MS/s and a TX at 3400 MHz. In session N1, it also accepted 914 and 433.92 MHz and refused 1000 MHz. In the twin, FN1 sent at 3.4 GHz, and PN1 or PN2 received over the CHEM 2-ray link. All ten recorder runs at 2, 11.52 and 15.36 MS/s had zero overflows, zero timeouts and no zero samples. The wall-clock rate at 2 MS/s was 2.0 MS/s. At PN1, the level was −81.0 dBFS with no TX, −47.6 dBFS with a TX of zeros and −21.95 dBFS with a tone (Fig. 7). CHEM did not stop outside the planned resets in the six radio sessions.

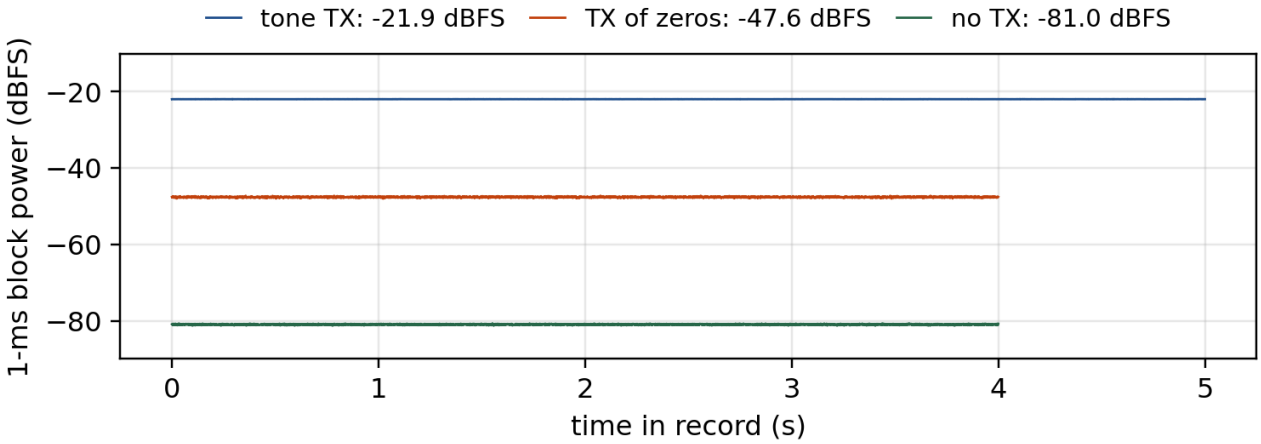


Figure 7: One-millisecond block power at PN1 for three continuous 2 MS/s records: a tone TX, a TX of zeros and no TX. Recorded again on 10 November 2026 (session N3).

Limits. The helper does not stop a retune that other code makes on the same device. Each RX read uses exactly `max_num_samps`. The UHD1 sample, which stopped CHEM, printed "Buffer size mismatch 2040 15360". We think that a read size that is not equal to `max_num_samps` can cause this message, but we did not test this.

6.2. Script 2: iq_recorder, SigMF Recorder with CHEM Link Log

File: `node_tools/aerpaw_py/iq_recorder.py` **Results:** `gnuradio_deploy/results/pyb_02`
Status: Tested in the twin (experiment X1532). **Couples with:** UHD2 sample (replaces it), all TX waveforms
Public start point: https://github.com/sigmf/sigmf-python

Purpose. `iq_recorder.py` writes one continuous V-USRP record as SigMF (`cf32_le`). The metadata has the core fields, the node data and the CHEM link state in an `aerpaw:` namespace. The recorder polls CHEM at 4 Hz during the record and writes the rows to a CSV file. It writes one annotation for each second with the CHEM distance, SNR, path loss, frequency offset and noise model. Therefore, each record of the twin becomes a labeled data set.

Coupling. The recorder replaces the UHD2 sample in the twin. Scripts 3, 4, 5, 7, 8 and 10 read its records. `dl_capture.py` uses it for the srsRAN and OCUDU cell records.

Command line.

```
python3 /root/claude_tools/aerpaw_py/iq_recorder.py /root/grdeploy/work_deployX/data/S2/tone_fo \
        --secs 30 --link 'M1>M3' --label tone_fo
```

Results. FN1 sent a 100 kHz tone at 3.4 GHz. PN1 recorded 30 s at 2 MS/s with the CHEM poll at 4 Hz. During the record, we changed the CHEM frequency offset from 0 to +300 and then to −200 Hz. The record held 60 000 000 samples, 122 CHEM rows and 30 annotations. The wall time was 30.0 s, with no overflow. The CHEM distance was 36.74 m, the SNR 25.32 dB and the path loss 77.43 dB. The library sigmf-python 1.14.0 accepted all 25 records of this campaign as valid SigMF. Fig. 8 shows the logged CHEM values.

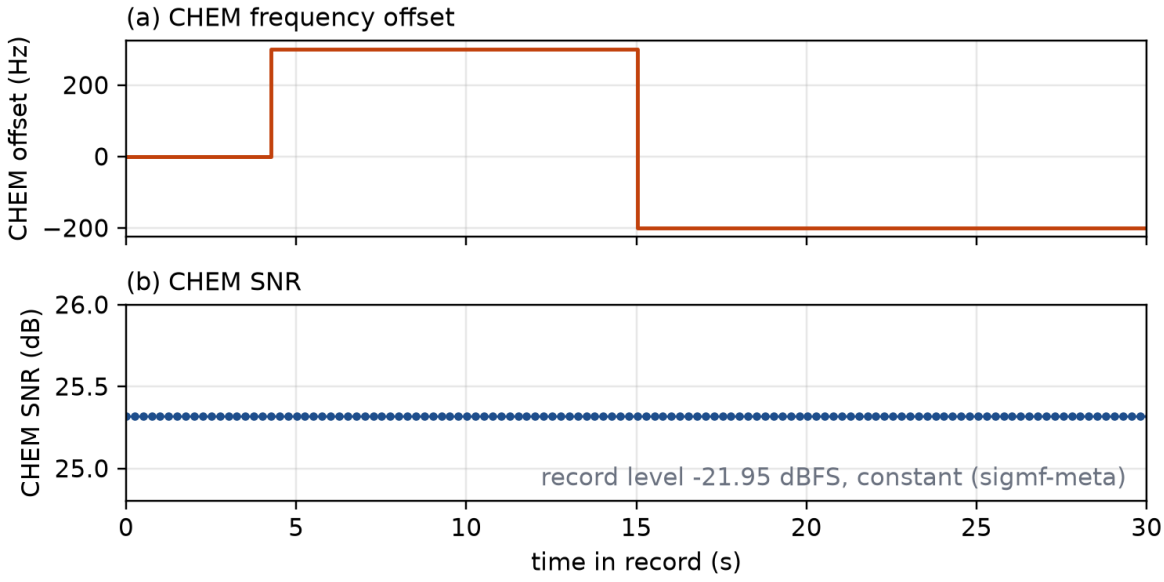


Figure 8: CHEM frequency offset (a) and CHEM SNR (b) that the recorder logged at 4 Hz during the 30-s record. The record level stayed at −21.95 dBFS.

Limits. sigmf-python is a private install (`pip -target` into the folder `_vendor`, no dependencies). It uses the system jsonschema 3.2. The `aerpaw:` namespace is our own extension. No public SigMF extension gives the CHEM link state. The CHEM rows use the clock of PN1.

6.3. Script 3: occupancy_monitor, Duty Cycle next to LTE and NR Cells

File: `node_tools/aerpaw_py/occupancy_monitor.py` **Results:** `gnuradio_deploy/results/pyb_03`
Status: Tested in the twin (experiment X1532). **Couples with:** srsRAN LTE (SE1 sample), OCUDU gNB with OAI NR UE, `siggen.py`
Public start point: https://github.com/NTIA/scos-actions

Purpose. `occupancy_monitor.py` measures the duty cycle of each channel in one captured band with an energy detector. It calculates the power of each channel in each block (default 1 ms). A block is occupied when the channel power is more than a threshold above the noise floor. The floor comes from a noise-only record (best) or from a low percentile of the median PSD. The output is a JSON file, a CSV time series and a figure.

Coupling. We used three sources. (a) `siggen.py` bursts from FN1 to PN1. (b) The srsRAN LTE DL of the SE1 sample (50 PRB, 3410 MHz, 11.52 MS/s), with srsEPC and srsENB on FN1 and srsUE on PN1. (c) The OCUDU n78 cell (3404.64 MHz, 10 MHz, 15.36 MS/s), with the gNB on FN1 and the OAI NR UE on PN1. PN2 monitored both cells, idle and with an iperf3 DL.

Command line.

```
python3 occupancy_monitor.py S1/bursts --channels=-750e3:250e3,-250e3:250e3,250e3:250e3,750e3:250e3 \
        --noise S1/noise_zeros --block-ms 0.5 --plot
python3 occupancy_monitor.py S3b/lte_load --rate 11.52e6 --nchan 10 --span 9e6 --thr-db 20 --plot
```

Results. For bursts with duty cycles of 1, 0.5, 0.25 and 0 in four channels, PN1 measured 1.0, 0.5, 0.3 and 0.0. The floor was −56.7 dBFS in 250 kHz. For the LTE cell in ten 900 kHz sub-bands and a threshold of 20 dB, the idle cell gave a duty of 0 to 0.2. The value 0.2 was in the two center sub-bands, and the lowest sub-band gave 0.06. An iperf3 DL at 35.1 Mbit/s gave 1.0 in all sub-bands. The mean level was −34.6 to −32.1 dBFS idle and −26.7 to −26.6 dBFS with load. For the OCUDU cell and a threshold of 10 dB, the duty was 0.00 to 0.06 idle. It was 0.72 to 1.00 with an iperf3 DL (Fig. 9).

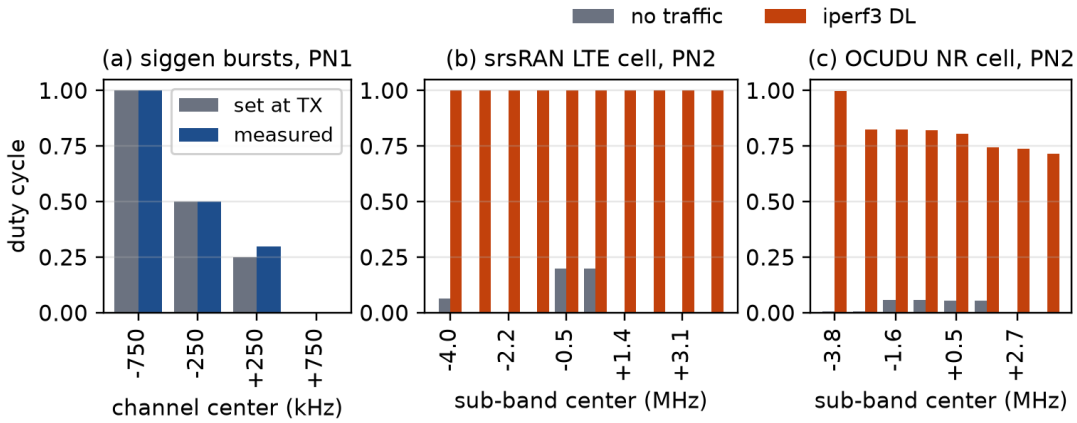


Figure 9: Duty cycle per channel: (a) the siggen bursts against the TX values, (b) the srsRAN LTE cell and (c) the OCUDU NR cell at PN2 without and with iperf3 traffic. Drawn again from the occupancy JSON files.

Limits. The value 0.30 for a duty of 0.25 comes from 0.5 ms blocks at the edges of the bursts. During the LTE iperf3 test, 35 % of the PN2 samples were at the −16 dBFS clip. Therefore, the load power is a lower limit. The NR load duty also includes the uplink of the UE on PN1 (TDD, same frequency). The LTE and NR floors are estimates (Appendix C).

6.4. Script 4: fixed_tune_scanner, Channelizer and Virtual Stepped Scan

File: `node_tools/aerpaw_py/fixed_tune_scanner.py` **Results:** `gnuradio_deploy/results/pyb_04`
Status: Tested in the twin (experiment X1532). **Couples with:** UHD1 sample (replaces its sweep), siggen bursts
Public start point: scos-actions stepped-frequency action (method only).

Purpose. `fixed_tune_scanner.py` cuts one tuned band into channels. For each channel, it uses a digital mixer, a low-pass filter and decimation (`scipy resample_poly`). Therefore, it never retunes the V-USRP. For each channel, it gives the power in dwell steps, the duty cycle against the noise floor and, as an option, the narrowband I/Q as SigMF. A "virtual stepped scan" visits the channels in turn, one dwell each, on the same record. It shows what a retune scanner such as UHD1 measures, without a retune.

Coupling. The scanner replaces the LO sweep of UHD1 inside one band. It used the PN1 record of the four-channel `siggen.py` bursts (duty 1, 0.5, 0.25 and 0, period 10 ms). The channels were at −750, −250, +250 and +750 kHz, with a bandwidth of 200 kHz and an output rate of 250 kS/s.

Command line.

```
python3 fixed_tune_scanner.py S1/bursts --channels=-750e3:200e3,-250e3:200e3,250e3:200e3,750e3:200e3 \
        --noise S1/noise_zeros --dwell-ms 0.5 --plot
```

Results. The full-time duty cycles were 1.00, 0.55, 0.30 and 0.00. A virtual stepped scan with a 2 ms revisit gave 1.00, 0.60, 0.20 and 0.00. With a 10 ms revisit, equal to the burst period, the scan gave 1.00, 1.00, 0.00 and 0.00. This alias error shows what a scanner that retunes measures on periodic traffic. The full-time values differ a little from Script 3 (0.5 and 0.3) on the same record. The reason is that the scanner uses 200 kHz channels after a filter, not 250 kHz fast Fourier transform (FFT) bins. The mean channel levels were −30.4, −33.4, −36.2 and −56.7 dBFS, and the floor was −58.4 dBFS. Fig. 10 shows the power of each channel.

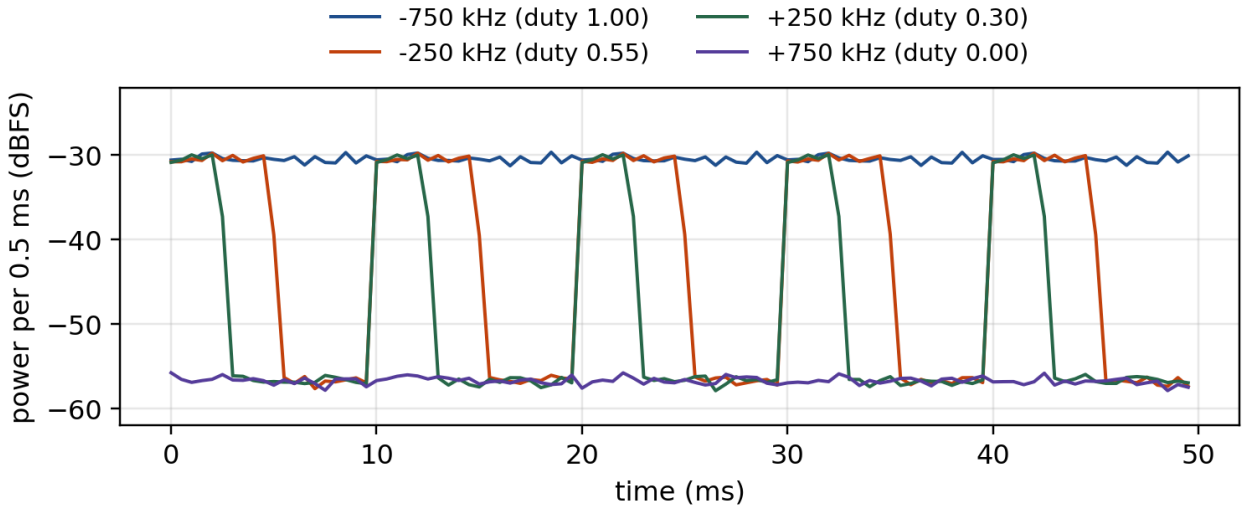


Figure 10: Power per 0.5 ms dwell in the four channels after the digital channelization of one 2 MS/s record.

Limits. We ran the scanner offline on a live record. The script also has a `--live` mode that records first. The virtual scan uses the same record, so it isolates the effect of the revisit time. All channels must be inside the tuned band, and the tool refuses other channels.

6.5. Script 5: channel_sounder, Zadoff-Chu Sounder and Power Delay Profile

File: `node_tools/aerpaw_py/channel_sounder.py` **Results:** `gnuradio_deploy/results/pyb_05`
Status: Tested in the twin (experiment X1532). **Couples with:** CHEM CIR settings, `chem_sweeper.py` (Script 11), integrated sensing and communication (ISAC) multipath studies (relative delays only)
Public start point: https://github.com/anilgurses/USRP-Channel-Sounder (method only).

Purpose. `channel_sounder.py` sends a 1023-chip m-sequence or a Zadoff-Chu sequence in one continuous stream. The receiver calculates the circular cross-correlation for each period and averages the power over the periods. The result is the power delay profile (PDP). The script gives the taps above a floor (−25 dB), the mean and rms delay spread, and the phase of the main tap in each period. The CHEM channel impulse response (CIR) is the reference.

Coupling. FN1 sent a Zadoff-Chu sequence (N 1021, root 7). On PN1, `chem_sweeper.py` applied the CHEM CIR tap sets clear, echo2, three and dense8 during its RX stream, with a settle time of 9 s. The sounder calculated the PDP of each record from 400 periods.

Command line.

```
python3 channel_sounder.py tx --seq zc --secs 110                                    # on FN1
python3 chem_sweeper.py cir clear,three,dense8,echo2 --link 'M1>M3' --settle 9 --secs 2 \
        --save-iq --out snd                                                           # on PN1
python3 channel_sounder.py ana snd_cir_three --seq zc --plot
```

Results. The profiles agreed with the pychem tap tables to 0.01 dB. The tap set echo2 gave −6.0 dB at 3 samples. The tap set three gave −7.96 and −13.98 dB at 2 and 5 samples. For dense8, the sounder found the four taps above −25 dB (−8.3, −10.63 and −18.21 dB at 1, 2 and 3 samples). The rms delay spread was 0.601 μs for echo2, 0.543 μs for three and 0.323 μs for dense8. The PDP noise was −55.4 dB. CHEM reported a tap vector of 0, 4, 6 and 8 samples for clear, echo2, three and dense8. This is the largest delay plus one.

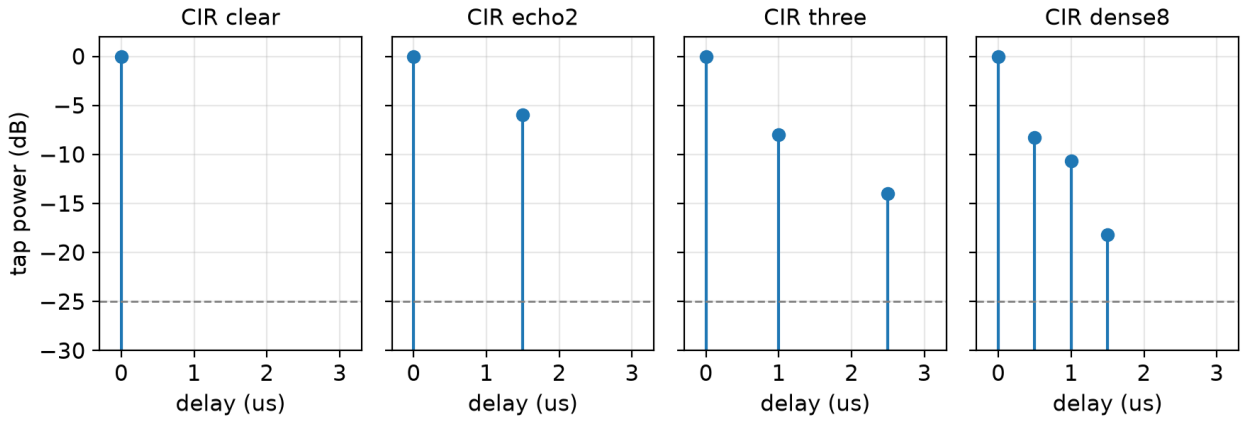


Figure 11: Measured PDP taps for four CHEM CIR settings at 2 MS/s (1 sample is 0.5 μs). The dashed line is the −25 dB floor. The taps agree with the pychem CIR tables to 0.01 dB. Drawn again from the result logs.

Limits. CHEM CIR taps are in samples, so the delays scale with the rate. The dense8 taps 4 to 7 are below the −25 dB floor. In session S1, the CIR commands came after the RX had stopped, and CHEM returned an empty list. A link exists only when both radios run. The delays are relative to the first tap. CHEM adds no delay for the distance, so the sounder cannot measure a range.

6.6. Script 6: siggen, Continuous Test-Signal Generator

File: `node_tools/aerpaw_py/siggen.py` **Results:** `gnuradio_deploy/results/pyb_06`
Status: Tested in the twin (experiment X1532). **Couples with:** all RX scripts, `meters.py` (known symbols)
Public start point:
https://github.com/EttusResearch/uhd/blob/master/host/examples/python/tx_waveforms.py

Purpose. `siggen.py` repeats one waveform period in one continuous V-USRP stream and fills the gaps with zeros. It has 11 kinds: tone, multitone (Schroeder phases), chirp, pn (1023-chip m-sequence), zc (Zadoff-Chu), noise, ofdm (OFDM-like), qpsk, 16qam, bursts and zeros. The kinds qpsk and 16qam are single-carrier signals with a root-raised-cosine (RRC) pulse and known symbols from a seed. The kind bursts gives noise bursts in sub-channels, with a duty cycle for each sub-channel. The kind zeros only registers the TX, so CHEM adds its noise at the RX.

Coupling. All RX scripts of this report use `siggen.py` as the transmitter. `meters.py` rebuilds the qpsk and 16qam symbols from the seed.

Command line.

```
python3 siggen.py qpsk --rms-dbfs -20 --secs 250
python3 siggen.py bursts --rms-dbfs -24 --secs 40
python3 siggen.py qpsk --save qpsk_ref.c64        # one period to a file, no radio
```

Results. FN1 sent seven kinds at 3.4 GHz and 2 MS/s, with a root mean square (RMS) level of −18 to −24 dBFS: zeros, tone, bursts, Zadoff-Chu, QPSK, 16QAM and OFDM-like. PN1 received each kind with the expected spectrum (Fig. 12). The tone was in the FFT bin at 100 006.1 Hz, the bin nearest to 100 kHz (bin width 30.5 Hz), 71.5 dB above the floor. The TX PAPR was 0.0 dB for the tone, 3.9 dB for QPSK, 5.6 dB for 16QAM, 10.6 dB for the OFDM-like signal and 12.1 dB for the bursts. With the kind zeros, CHEM added noise at −47.6 dBFS at PN1. We built multitone, chirp, pn and noise offline only (`--save`). We did not test `--gain-sched` on air.

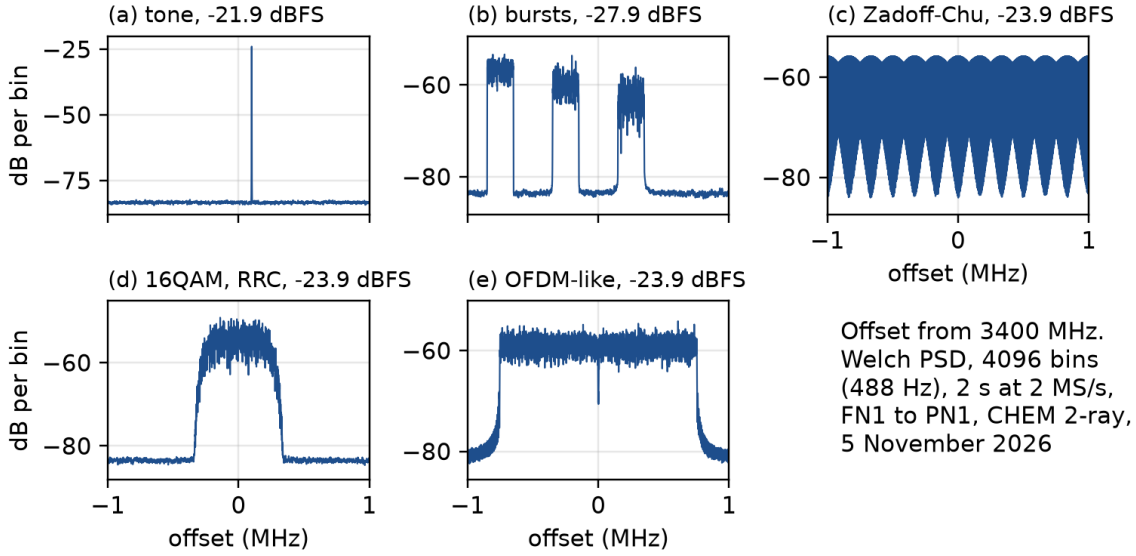


Figure 12: Spectra at PN1 of five siggen signals sent from FN1 at 3.4 GHz and 2 MS/s over the CHEM 2-ray link. Recorded again on 5 November 2026 (session R3) with the settings of 4 November, for a readable figure. The periodic Zadoff-Chu signal gives spectral lines 1.96 kHz apart. At 488 Hz per bin, the lines merge into a regular pattern.

6.7. Script 7: meters, SNR, EVM and PAPR Meters

File: `node_tools/aerpaw_py/meters.py` **Results:** `gnuradio_deploy/results/pyb_07`
Status: Tested in the twin (experiment X1532). **Couples with:** CHEM fixed SNR, Scripts 2, 5, 6 and 11
Public start point: https://github.com/mwickert/scikit-dsp-comm, https://github.com/mhostetter/sdr (methods only).

Purpose. `meters.py` gives three SNR values, an EVM and a PAPR CCDF for one record. The data-aided SNR fits the known periodic TX waveform to the record over the full sample band. The noise-reference SNR compares the record with a noise-only record. The spectral SNR compares the in-band power with the out-of-band noise density. The EVM uses an RRC matched filter and the symbols from the siggen seed. The script also gives the share of samples near the −16 dBFS clip.

Coupling. FN1 sent RRC QPSK and 16QAM (roll-off 0.35, 4 samples per symbol, 2048-symbol period). CHEM set a fixed SNR of 0 to 30 dB, or the link budget, on the FN1 to PN1 link. `chem_sweeper.py` ran the meters on each 2 s record.

Command line.

```
python3 meters.py sweep_16qam_noise_LINK_BUDGET --ref qam16_ref.c64 --mod 16qam --sig-bw 675e3 \
        --plot meters.png
```

Results. From 0 to 30 dB, the data-aided SNR agreed with the CHEM value to 0.01 dB. The SNR from the EVM was 5.98 to 6.06 dB higher. This agrees with the 6.02 dB matched-filter gain at 4 samples per symbol. The QPSK EVM was 1.59 % at 30 dB, 4.98 % at 20 dB, 15.75 % at 10 dB and 49.78 % at 0 dB. At the CHEM link budget (23.44 dB), the data-aided SNR was 23.44 dB. 16QAM had an EVM of 3.37 % and a PAPR of 5.25 dB at a CCDF of 0.001 (0.25-dB grid) (Fig. 13). The spectral full-band SNR at 15 dB was 14.98 dB. In a synthetic check at 10.0 dB, the data-aided SNR was 10.0 dB and the spectral SNR 10.01 dB.

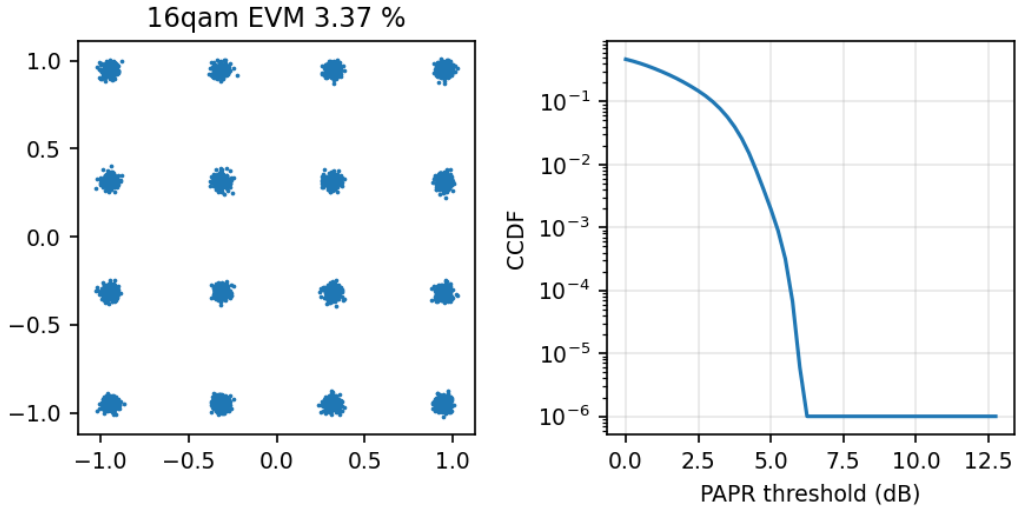


Figure 13: 16QAM constellation after the matched filter and PAPR CCDF of the PN1 record at the CHEM link budget (23.4 dB). The flat line at 10^{-6} is the lower limit of the plot.

Limits. The CHEM fixed SNR refers to the full 2 MHz sample band. The RX signal power stays constant, and CHEM adds noise (−23.9 dBFS signal, −21.0 dBFS total at 0 dB). The noise-reference SNR of a TX of zeros against no TX was 33.4 dB, because CHEM adds noise only with a registered TX. The CHEM value is not the SNR in the signal band or after the matched filter (see Section 2).

6.8. Script 8: dl_capture, LTE and NR Downlink Capture and Decode

File: `node_tools/aerpaw_py/dl_capture.py` **Results:** `gnuradio_deploy/results/pyb_08`
Status: Tested in the twin (experiment X1532). **Couples with:** srsRAN LTE eNB, OCUDU gNB, nr-sib1 (Rust)
Public start point: https://github.com/zackiles/nr-sib1

Purpose. `dl_capture.py` records a cell downlink at the cell rate and center frequency with `iq_recorder.py`. Then it checks the primary synchronization signal (PSS): the three NR m-sequence PSS or the three LTE Zadoff-Chu PSS. It gives the best NID2 (the PSS part of the cell identity), the normalized correlation peak, the number of peaks and the median spacing. The mode `nrsib1` cuts the record into windows and runs the Rust nr-sib1 decoder on each window. The decoder finds the synchronization signal block (SSB), the master information block (MIB) and the system information block 1 (SIB1).

Coupling. FN1 ran the srsRAN eNB or the OCUDU gNB with TX gain 70. PN2 (43.7 m) recorded the srsRAN LTE DL (3410 MHz, 11.52 MS/s) and 0.6 s of the OCUDU n78 DL (3404.64 MHz, 15.36 MS/s). We built the nr-sib1 binary in an earlier session (folder `gnuradio_deploy/work/deployL`).

Command line.

```
python3 dl_capture.py capture nr_dl --rat nr --rate 15.36e6 --freq 3404.64e6 --secs 0.6
python3 dl_capture.py nrsib1 nr_dl --decoder /root/grdeploy/work_deployX/bin/nrsib1_decode
python3 dl_capture.py capture lte_idle --rat lte --rate 11.52e6 --freq 3410e6 --secs 2
```

Results. For the srsRAN eNB, PN2 found NID2 1 with a correlation of 0.988, and 40 PSS peaks in 0.2 s at a spacing of 5.0 ms. For the OCUDU gNB, PN2 found NID2 1 with a correlation of 0.998 and a spacing of 10.0 ms (Fig. 14). The nr-sib1 decoder found the SSB sync and the MIB in all 26 windows of 100 ms, and a CRC-valid SIB1 in 16 windows. The sync quality was 17.3 dB, and the system frame number (SFN) was 959 to 1009. The SIB1 gave physical cell identity (PCI) 1, public land mobile network (PLMN) 001/01 and tracking area code (TAC) 7. It also gave cell identity 6733824, 24 PRB and physical random-access channel (PRACH) configuration index 159. It gave a 5 ms TDD pattern (6 DL slots, 8 DL symbols and 3 UL slots). The decoder reported a CFO of −3502 Hz.

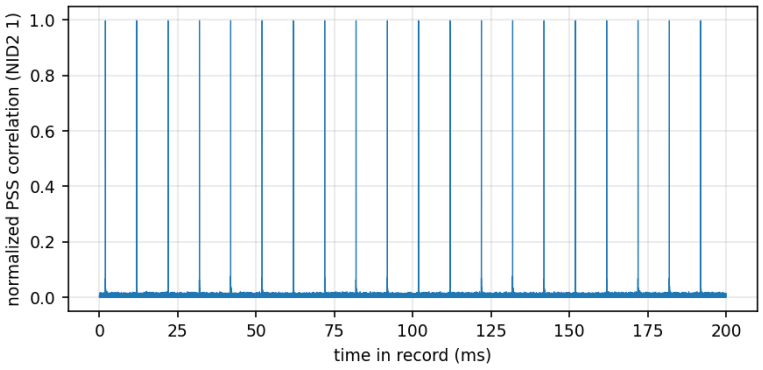


Figure 14: Normalized NR PSS correlation (NID2 1) over the first 0.2 s of the OCUDU downlink record at PN2.

Limits. SIB1 needs the Type0-PDCCH occasion inside the window, so 10 windows had no SIB1. The −3.5 kHz CFO is the decoder value, the same as in the earlier test (−3504 Hz), but CHEM reports a frequency offset of 0. In the LTE record, 4.9 % of the samples were at the −16 dBFS clip. The NR count of 20 peaks in 0.2 s includes some secondary peaks, but the median spacing is 10 ms.

6.9. Script 9: link_logger, CHEM, Ping and iperf3 in One CSV

File: `node_tools/aerpaw_py/link_logger.py` **Results:** `gnuradio_deploy/results/pyb_09`
Status: Tested in the twin (experiment X1532). **Couples with:** OCUDU gNB, Open5GS, OAI NR UE, iperf3, ping
Public start point: our earlier `chem_logger.py` (Report-1).

Purpose. `link_logger.py` joins the CHEM link state, the ping round-trip time (RTT) and the iperf3 throughput in one time-stamped CSV on one node. It has three threads: a CHEM poll, a ping and an iperf3 client with one row for each 1 s interval. It does not use the radio. It writes a long CSV, a CSV with one row for each second, a JSON summary and a figure.

Coupling. FN1 ran the OCUDU gNB and Open5GS. The OAI NR UE (`nr-uesoftmodem`) on PN1 attached at the first start (n78, 3404.64 MHz, 10 MHz, 15.36 MS/s, gNB TX gain 70). The logger on FN1 ran for 45 s, with a ping every 0.5 s and a 25 s iperf3 TCP DL to the UE.

Command line.

```
python3 link_logger.py --target <UE_IP> --secs 45 --iperf-secs 25 --iperf-delay 5 \
        --link 'M1>M3' --plot
```

Results. The receiver throughput was 4.59 Mbit/s, and the sender throughput was 5.45 Mbit/s. Seven of 25 intervals had 0 Mbit/s. The median ping round-trip time (RTT) was 24.6 ms without traffic and 1114 ms during the TCP DL, with a maximum of 1530 ms (per-ping values, `result.json`). 38 of 128 pings got no reply. The median CHEM SNR on the FN1 to PN1 link was 30.57 dB. The CSV had 389 rows. Fig. 15 shows the three time series.

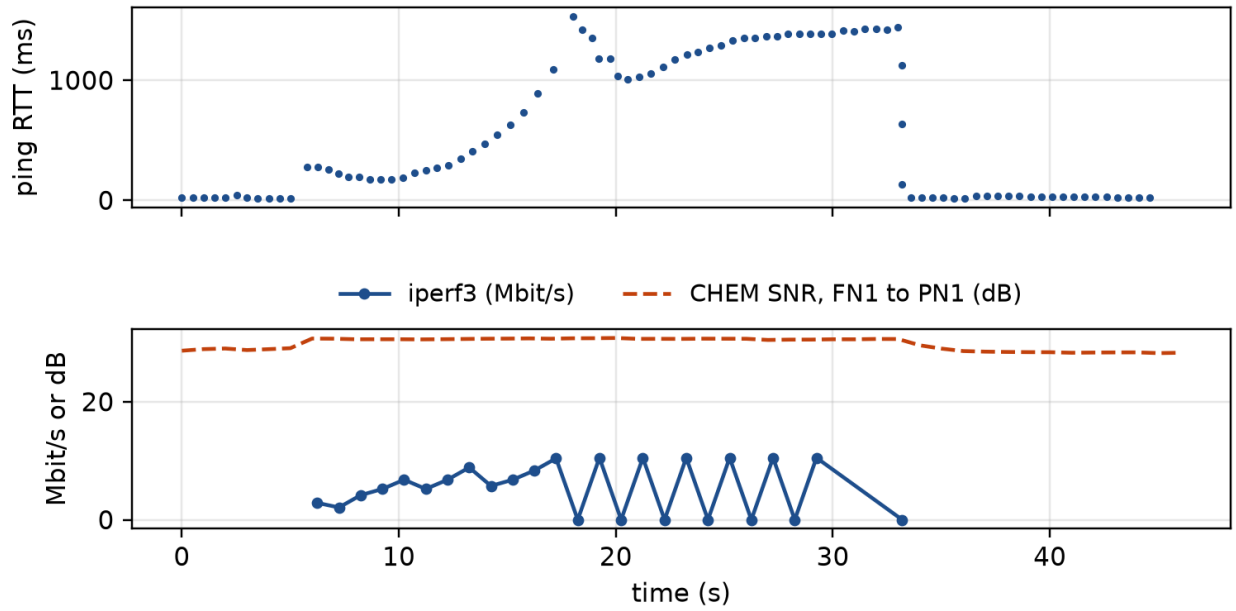


Figure 15: Ping RTT, iperf3 throughput per second and CHEM SNR from one link_logger run during an OCUDU TCP downlink test.

Limits. In the first run (session S4), the parser did not read the intervals with 0 bits/sec. Therefore, it reported 3.68 Mbit/s over 10 intervals. We corrected the parser, added the receiver line and a raw iperf3 file, and did the test again (S4b). The RTT increase under TCP load points to a queue. We did not study the cause of the intervals with zero throughput. The clocks of FN1 and PN1 differ by approximately 25 ms, so the logger time-stamps all data on one node.

6.10. Script 10: doppler_cfo, Pilot-Tone CFO and Doppler Tracker

File: `node_tools/aerpaw_py/doppler_cfo.py` **Results:** `gnuradio_deploy/results/pyb_10`
Status: Tested in the twin (experiment X1532). **Couples with:** CHEM frequency offset and Doppler, `iq_recorder.py` CHEM log
Public start point: PySDR frequency synchronization (method only), https://pysdr.org.

Purpose. `doppler_cfo.py` tracks the carrier frequency offset (CFO) of a pilot tone in windows (default 100 ms). It uses a Hann window, a zero-padded FFT, a peak search near f_0 and a quadratic interpolation. It compares the result with the CHEM frequency offset and Doppler from the link log of the recorder. It gives the error after each CHEM change and the time from the change to the measured step.

Coupling. FN1 sent a 100 kHz tone with `siggen.py`. PN1 recorded 30 s with `iq_recorder.py` (the record of Script 2). During the record, we set the CHEM frequency offset on FN1 to PN1 to +300 Hz and then to −200 Hz.

Command line.

```
python3 doppler_cfo.py S2/tone_fo --f0 100e3 --plot
```

Results. The measured offsets in the three segments were 0.00, +300.00 and −200.00 Hz, with a standard deviation below 0.01 Hz in each segment. The median error against the settled CHEM value was 0.001 Hz over 300 windows. Each step in the record came within 0.2 s of the change in the CHEM link log (−0.01 s and −0.18 s). With the CHEM offset at 0 Hz, the tone had no offset. Therefore, the twin adds no CFO of its own on this link. Fig. 16 shows the offset of each window.

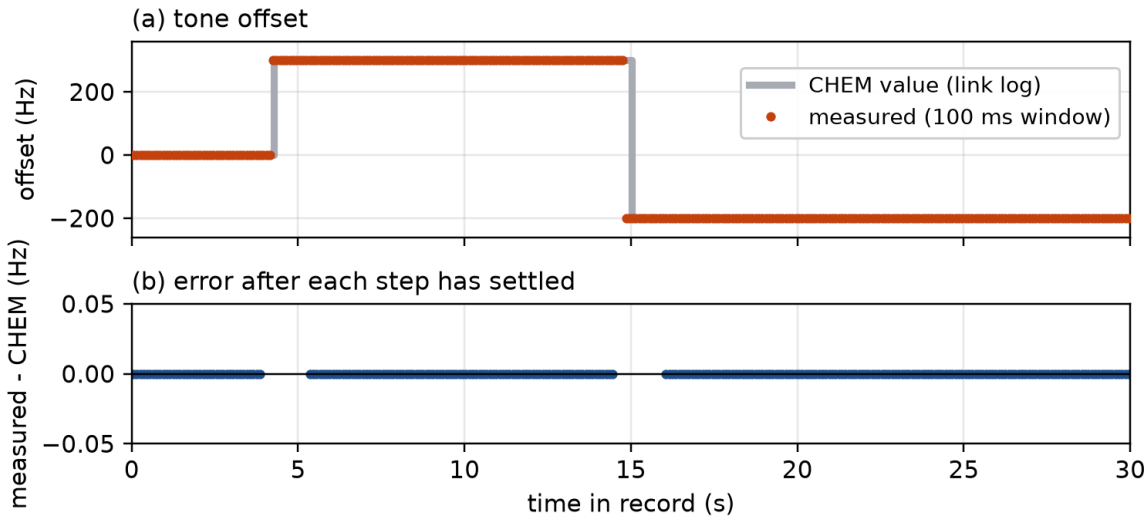


Figure 16: Measured tone offset per 100 ms window and the CHEM frequency-offset value from the link log (a), and the error after each step has settled (b).

Limits. The drone was on the ground, and the CHEM Doppler was 0 Hz. Therefore, this test checks only the CFO path, not a Doppler shift from a flight. The CHEM log changes when CHEM applies the setting. With node clocks, the offset acted 0.53 s after the command (R3). Version 1 gave approximately 7 s, from the clock of the control computer. The three offsets fall on the 1.25-Hz grid of the zero-padded FFT, where the interpolation has no bias. For other offsets, the bias of the interpolation can be up to a few hundredths of a hertz.

6.11. Script 11: chem_sweeper, CHEM Sweep with Automatic Capture

File: `node_tools/aerpaw_py/chem_sweeper.py` **Results:** `gnuradio_deploy/results/pyb_11`
Status: Tested in the twin (experiment X1532). **Couples with:** CHEM fixed SNR, frequency offset and CIR, Scripts 5, 6 and 7
Public start point: none (the tool is specific to CHEM).

Purpose. `chem_sweeper.py` applies one CHEM value at a time to one link, waits, reads the CHEM link state, records and measures. It runs on the RX node, and `siggen.py` runs on the TX node. One RX stream serves the full sweep. For each point, it writes one CSV and JSON row with the CHEM values, the data-aided SNR and the EVM. At the end, it sets the link back to the value of `--restore`. It is the data-set factory of the twin.

Coupling. FN1 sent QPSK. The sweeper on PN1 set a fixed SNR of 30, 25, 20, 15, 10, 5 and 0 dB, then the link budget, on the FN1 to PN1 link. It waited 16 s (the default of version 1) and recorded 2 s for each point, and it ran `meters.py`. A 16QAM sweep (link budget, 25 and 15 dB) and the CIR sweep of Script 5 used the same tool.

Command line.

```
python3 chem_sweeper.py noise 30,25,20,15,10,5,0,LINK_BUDGET --link 'M1>M3' --ref qpsk_ref.c64 \
        --mod qpsk --restore LINK_BUDGET --out sweep_qpsk
```

Results. The QPSK sweep had 8 points, and each point took 18.2 s. CHEM read back the set SNR at each point. The data-aided SNR was 30.0, 25.01, 20.0, 15.0, 10.01, 5.0 and 0.01 dB, and 23.45 dB at the link budget. Therefore, the error was 0.01 dB or less at all 8 points. The EVM was 1.59, 2.81, 4.98, 8.85, 15.75, 28.02 and 49.78 %, and 3.35 % at the link budget. Fig. 17 shows the QPSK and 16QAM sweeps.

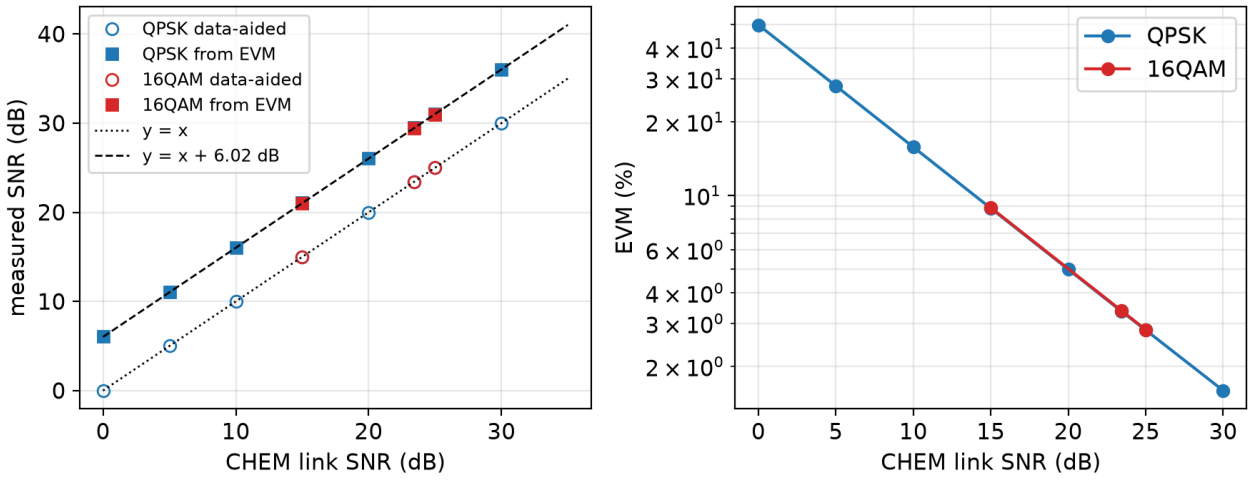


Figure 17: Data-aided and EVM-based SNR (left) and EVM (right) against the CHEM link SNR for the QPSK and 16QAM sweeps. Drawn again from the sweep CSV files.

Limits. The settle time of 16 s came from an earlier test that paired the noise steps of a capture with the wrong commands (Section 7.3). With a settle time of 3 s, the error stayed within 0.04 dB, and a point took 5.27 s (R1, Section 7.1). Version 2 of the script uses 3 s as the default. CHEM settings apply only to links that exist when the command runs. Therefore, the sweeper keeps its RX stream open for the full sweep.

6.12. Script 12: chem_watchdog, CHEM Port Watchdog with reset_chem

File: `node_tools/aerpaw_py/chem_watchdog.py` **Results:** `gnuradio_deploy/results/pyb_12`
Status: Tested in the twin (experiment X1532). **Couples with:** CHEM, `reset_chem`, `/root/reset.sh`
Public start point: none (the tool is specific to CHEM).

Purpose. `chem_watchdog.py` opens TCP connections to the CHEM ports 5000 (control) and 6000 (V-USRP configuration) at a fixed period. When both checks fail for longer than a grace time, it logs an incident. As an option, it stops the local radio programs. Then it runs `reset_chem` once and measures the time until both ports answer again. It also reads the CHEM log counters ("TX queue is full" and "Drops"). The option `--max-resets` limits the actions, and `--dry-run` only logs.

Coupling. The watchdog ran on FN1 during `/root/reset.sh`. In session S1, the period was 3 s and the grace time 20 s (log only). In session S2, the period was 2 s and the grace time 5 s (action with `reset_chem`).

Command line.

```
python3 chem_watchdog.py --grace 5 --period 2 --secs 180 --max-resets 1 --out watchdog
```

Results. In session S1, both ports stopped for 12.0 s during `/root/reset.sh`, and the 20 s grace time kept the watchdog passive. In session S2, the watchdog ran `reset_chem` 6.0 s after the stop. The reset took 15.9 s, and both ports answered again 26.9 s after the stop. The CHEM log had no "TX queue is full" line. The number of drop lines went from 4 to 9 during S1. Fig. 18 shows the events.

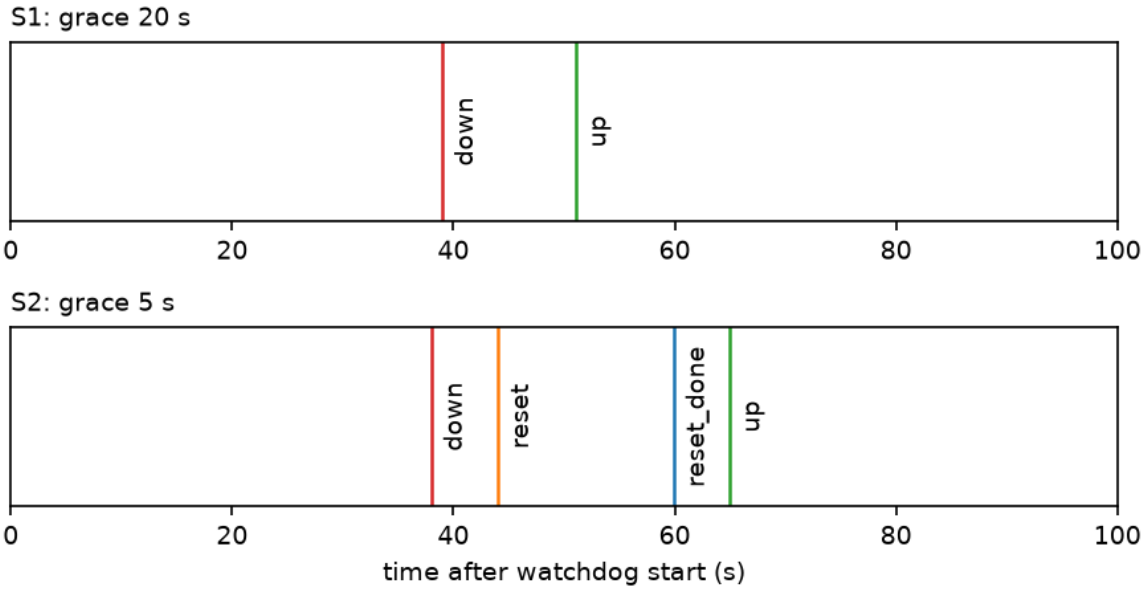


Figure 18: Watchdog events during the experiment resets of sessions S1 (grace 20 s) and S2 (grace 5 s).

Limits. The action in S2 ran during the CHEM restart of `/root/reset.sh`, on purpose, with the radio lock. No unplanned CHEM stop occurred in this campaign. Therefore, the watchdog did not see a real CHEM crash. The port stop of 12 to 27 s is shorter than the 55 s wait of the reset helper. CHEM is shared by all users. Therefore, set a grace time above the reset time unless you want an action.

6.13. Script 13: preflight, V-USRP Self-Test

File: `node_tools/aerpaw_py/preflight.py` **Results:** `gnuradio_deploy/results/pyb_13`
Status: Tested in the twin (experiment X1532). **Couples with:** all scripts (run before each campaign)
Public start point: Ettus `benchmark_rate.py` (method only).

Purpose. `preflight.py` checks a node before a campaign. The mode `check` looks at the environment file, the CHEM ports, the V-USRP (`uhd_find_devices`) and the accepted rates. It also checks that the code rules refuse 0.5 MS/s and a TX at 3.5 GHz, without a radio call. The mode `tx` sends a continuous tone. The mode `rx` records and checks seven items: the CHEM link, the tone frequency, the tone-to-floor ratio, the sample rate, no zero samples, no overflow and the RX level. It prints PASS or FAIL for each item.

Coupling. Run it before every other script. FN1 sent the 100 kHz tone, and PN1 received for 5 s.

Command line.

```
python3 preflight.py check
python3 preflight.py tx --secs 40      # on FN1
python3 preflight.py rx --secs 5       # on PN1
```

Results. The check passed on FN1 and PN1. `uhd_find_devices` listed the b205mini of each node. The V-USRP accepted 1, 2 and 23.04 MS/s and reported a master clock rate of 0.001 Hz. In the tone loop from FN1 to PN1 at 3.4 GHz, all seven items passed. CHEM showed an SNR of 25.3 dB. The tone was in the FFT bin at 100 006.1 Hz, the bin nearest to 100 kHz (bin width 30.5 Hz), 71.5 dB above the floor, at a level of −21.95 dBFS. The rate was 2.000 MS/s with no overflow (Fig. 19).

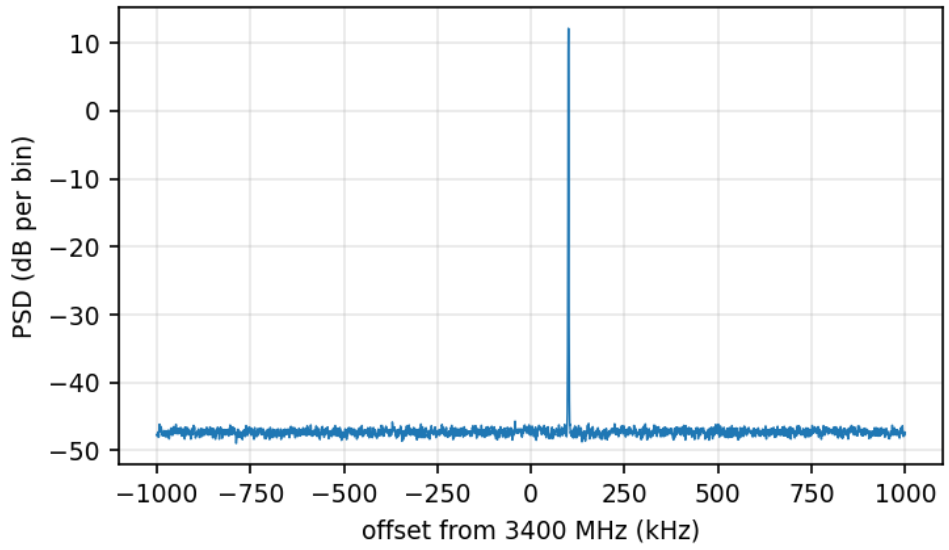


Figure 19: Spectrum at PN1 of the preflight tone (100 kHz) from FN1 at 3.4 GHz and 2 MS/s.

Limits. `uhd_find_devices` also lists the serial AERPAW-B210-01 from the V-USRP configuration file. The check opens the device. Therefore, wait 20 s before the next radio program on that node. The master clock rate of 0.001 Hz agrees with our earlier CHEM tests (finding L23). Do not use it to calculate a rate.

6.14. Script 14: mat2sigmf, UHD2 Snapshots to SigMF

File: `node_tools/aerpaw_py/mat2sigmf.py` **Results:** `gnuradio_deploy/results/pyb_14`
Status: Tested in the twin (experiment X1532). **Couples with:** AERPAW UHD2 sample (`get_IQ.py`), `siggen.py`
Public start point: https://github.com/sigmf/sigmf-python

Purpose. `mat2sigmf.py` converts the `.mat` snapshots of the AERPAW UHD2 sample (`results_<LO>_<fs>_<time>.mat`) and raw complex64 files to SigMF. The snapshots of one LO and rate become one `cf32_le` record. The record has one capture segment and one annotation (RMS level) for each snapshot. The time comes from the file name (America/New_York). As an option, the tool adds the CHEM rows of the record time as annotations. It validates each output with sigmf-python.

Coupling. We ran a copy of the UHD2 sample `get_IQ.py` on PN1 with LOs of 3400 and 3410 MHz and a total duration of 60 s. The shipped `myConstants.py` was not changed. FN1 sent a 1.5 MHz OFDM-like signal at 3400 MHz and −20 dBFS with `siggen.py`.

Command line.

```
python3 get_IQ.py        # copy of SDR_control/LTE_IQ with fLO = (3400e6, 3410e6), total_duration = 60
python3 mat2sigmf.py mat 'uhd2_mat/results_*.mat' --out-dir uhd2_sigmf
```

Results. The sample wrote 172 snapshots of 20 ms, 86 for each LO, with 40 000 samples each. The snapshot period for each LO was 0.69 s. The tool made one valid SigMF record for each LO (2 of 2). At 3400 MHz, the median snapshot level was −28.2 dBFS, and the signal band was 21.1 dB above the band edge. At 3410 MHz, the level was −51.9 dBFS, as in our earlier Test 4 on 31 October, with a flat spectrum (noise only, 0.0 dB in-band to edge). Fig. 20 shows the level of each snapshot.

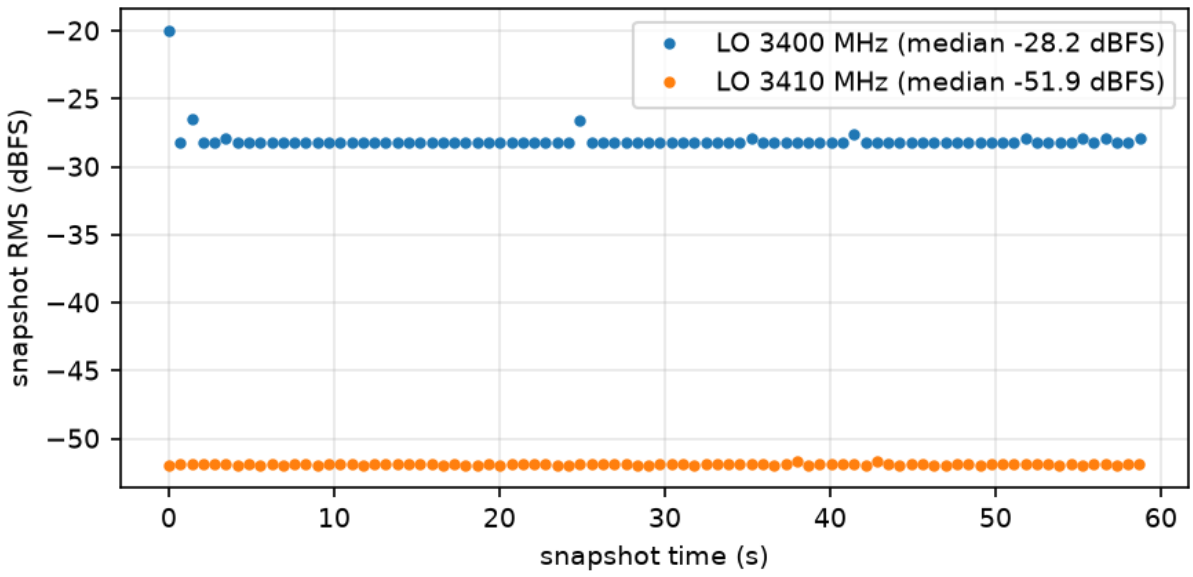


Figure 20: RMS level of each UHD2 snapshot after the conversion, for the two LOs of `get_IQ.py`.

Limits. We did not run the UHD1 sample, because it stops CHEM. The flat spectrum at 3410 MHz supports the explanation that the off-channel level of Test 4 is noise. We do not know why this level (−51.9 dBFS) is below the CHEM noise at 3400 MHz (−47.6 dBFS). The UHD2 snapshots at an LO that is not the TX frequency hold no CHEM signal. The converter keeps the LO of each capture segment, so that a reader can separate them.

7. New Twin Experiments (5 and 10 November 2026)

Version 1 of this report had two gaps. It gave no recipe for a data set, although the twin labels each record exactly. It also did not connect the scripts with the live metrics of the 5G stacks (Section 2.7). In addition, its CHEM delays came from the clock of the control computer, which was 13.3 s behind the node clocks. We therefore did three small experiments on 5 November 2026 (R1 to R3). Each session held the radio lock on FN1 from start to end and began with `/root/reset.sh` and the 2-ray channel. The result folders are `pyb_15` to `pyb_17`, and the commands are in Section 10. After the review of 10 November 2026, we did four more experiments (N1 to N4) with the same session frame. Their result folders are `pyb_18` to `pyb_21`.

7.1. R1: a labeled SigMF data set at several CHEM SNRs

Why. Machine learning (ML) work on modulation classification or SNR estimation needs many records with exact labels. In the twin, CHEM sets the SNR of the link, and `siggen.py` knows the modulation and the symbols. Each record can therefore carry exact labels.

Method. FN1 sent QPSK, and then 16QAM, at −20 dBFS with `siggen.py`. On PN1, `chem_sweeper.py` set the CHEM fixed SNR to 30, 0, 20, 5, 25, 10 and 15 dB. This order makes each step large. It waited 3 s after each command instead of 16 s, recorded 2 s and wrote one SigMF record for each point (`--save-iq`). The new offline tool `ds_manifest.py` then joined the sweep CSV and the SigMF files. It wrote a manifest with one row for each record: file, label, CHEM SNR, measured SNR, EVM, level and start time. It also added one annotation with these labels to each `.sigmf-meta` file. Before the sweeps, PN1 recorded 24 s, and we sent two fixed-SNR commands during this record. We time-stamped the commands on FN1 and compared them with the record (PN1 clock).

Results. Fig. 21 shows the results.

- The fixed SNR acted 0.55 s and 0.49 s after the time stamp of the command. The CHEM link log showed the new value after 0.75 s and 0.53 s. The record level went from −23.94 dBFS at 30 dB to −20.95 dBFS at 0 dB, because CHEM adds noise with the power of the signal.
- With a settle time of 3 s, the data-aided SNR agreed with the CHEM SNR within 0.04 dB at all 14 points (QPSK and 16QAM). This is the same accuracy as the 16-s sweep of Script 11. The QPSK EVM was 1.59 % at 30 dB and 49.84 % at 0 dB, as in Script 11.
- One point took 5.27 s instead of 18.2 s. The 14 points of the two sweeps took 74 s, without the start of the radios.
- `ds_manifest.py` labeled 14 records and flagged none (all errors below 0.5 dB). Each 2-s record at 2 MS/s is 32 MB.

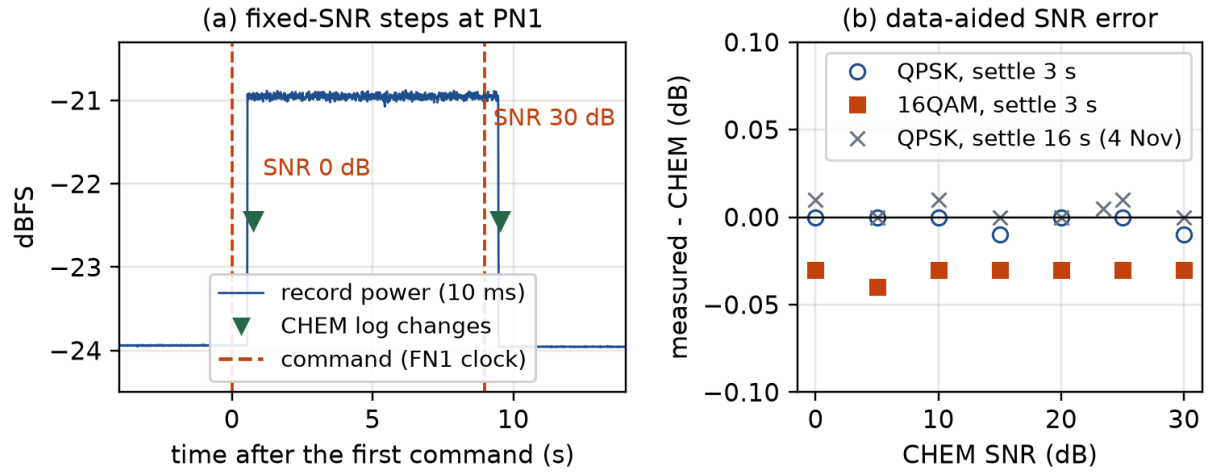


Figure 21: R1. (a) Record power at PN1 during two CHEM fixed-SNR commands (time stamps on FN1). The triangles show when the CHEM link log changed. (b) Data-aided SNR minus CHEM SNR for the sweeps with a 3-s settle time and for the 16-s QPSK sweep of Script 11.

Recipe for a data set. (1) Take the radio lock, reset, and set the channel. (2) Write the reference period of each waveform (`siggen.py KIND -save`). (3) For each waveform, start `siggen.py` on the TX node and run `chem_sweeper.py noise <SNR list> -settle 3 -secs 2 -save-iq` on the RX node. Use a CIR or a frequency-offset sweep in the same way (`cir`, `fo`). (4) Run `ds_manifest.py -annotate` on the CSV files. (5) Copy the SigMF files and the manifest from the node, and delete them there. To add the multipath or the CFO as labels, record the same SNR list for each CHEM setting.

Limits. We tested only additive white Gaussian noise (AWGN) steps on one ground link at 2 MS/s, with two single-carrier waveforms. The 3-s settle time is for the fixed SNR. We did not test it for a CIR or a frequency offset in a sweep (R3 gives their delay). At 0 dB, 1.4 % of the samples of the step record were near the clip. Keep the TX level at −20 dBFS or lower for low-SNR points. The label is the CHEM value over the full sample band, not over the signal band.

7.2. R2: a spectrum monitor next to a live OCUDU cell, with the gNB metrics

Why. Script 3 measured the duty cycle of an NR cell from outside. It could not tell how much of the cell the gNB really used. The OCUDU WebSocket metrics give this value. A comparison shows what an energy detector can and cannot see, and the gNB values can label a spectrum data set.

Method. The OCUDU gNB and Open5GS ran on FN1 with the WebSocket metrics on (`gnb_base.yaml` of Report-2, port 8001). The configuration was 10 MHz (24 PRBs), band n78 at 3404.64 MHz and a 5-ms TDD pattern. The OAI NR UE on PN1 attached after 8.7 s. On FN1, `ws_tool.py metrics` logged one JSON message each second. We offered iperf3 user datagram protocol (UDP) to the UE at 0, 2, 8 and 25 Mbit/s. During each load, PN2 recorded 2 s at 15.36 MS/s. Before the cell started, PN2 recorded a TX of zeros at the same frequency and rate as the noise floor. The helper `r6_ana.py fold` ran the analysis of `occupancy_monitor.py` with 8 sub-bands of 1.08 MHz, blocks of 0.5 ms (one slot) and a threshold of 10 dB. It also folded the occupied blocks over the 5-ms TDD period. For each record, we took the mean of the gNB messages in the same seconds. The values are the DL MCS, the DL bit rate and the physical downlink shared channel (PDSCH) PRBs for each slot of the TDD period.

Results. Table 5 and Fig. 22 give the results.

Table 5: R2: monitor on PN2 and gNB metrics for four DL loads (single run, 2-s records, gNB values from the 1-s WebSocket messages in the same seconds). PRB use: used PDSCH PRBs divided by 24 PRBs times 10 slots.

UDP offered (Mbit/s)	0	2	8	25
Received at the UE (Mbit/s)	–	2.00	7.99	15.5
gNB DL bit rate (Mbit/s)	0	2.15 to 2.16	8.54 to 8.59	10.65 to 19.35
gNB DL MCS	–	28	27 to 28	15 to 28
gNB DL PRB use, all slots	0.00	0.07	0.29	0.65
Monitor duty, mean of 8 sub-bands	0.05	0.14	0.44	0.60
Monitor duty, lowest sub-band	0.05	0.22	0.62	0.88
Monitor level (dBFS)	−31.4	−29.2	−24.9	−26.6

- The monitor duty increased with the load, as the PRB use of the gNB did. At full buffer, the two values were close: 0.60 and 0.65.
- At low load, the monitor gave 1.5 to 2 times the PRB use (0.14 against 0.07 at 2 Mbit/s, 0.44 against 0.29 at 8 Mbit/s). The energy detector marks a 1.08-MHz sub-band as occupied for the full slot when only some of its resource elements carry energy. The lowest sub-band had the highest duty at each load. A probable cause is the control region (PDCCH), which the gNB sends in each scheduled slot.
- The idle cell gave a duty of 0.05 in all sub-bands. The fold put these blocks in one slot of every second 5-ms period, which agrees with the synchronization and system-information bursts. The gNB reported 0 PRBs.
- At full buffer, the fold showed the TDD pattern: five slots at 0.99 to 1.00, one slot at 0.56, three slots at 0.12 to 0.17 and one slot at 0. The gNB reported 23 to 24 PRBs in six DL slots, 13 PRBs in one slot and no PDSCH in three slots. The low values in the last slots are the uplink of the UE on PN1 (same frequency), which PN2 also receives.
- The monitor duty cannot go above the DL share of the TDD pattern. Therefore, a duty of 0.6 means a full cell, not a cell at 60 % load.
- The noise floor of the TX of zeros at 15.36 MS/s was −38.76 dBFS at PN2. This is 8.9 dB above the level at 2 MS/s at PN1 (−47.6 dBFS). The ratio of the rates is also 8.9 dB ($10 \log_{10}(15.36/2)$). The two floors come from different nodes. Session N3 (Section 7.7) repeated the test on one node and found the same ratio.

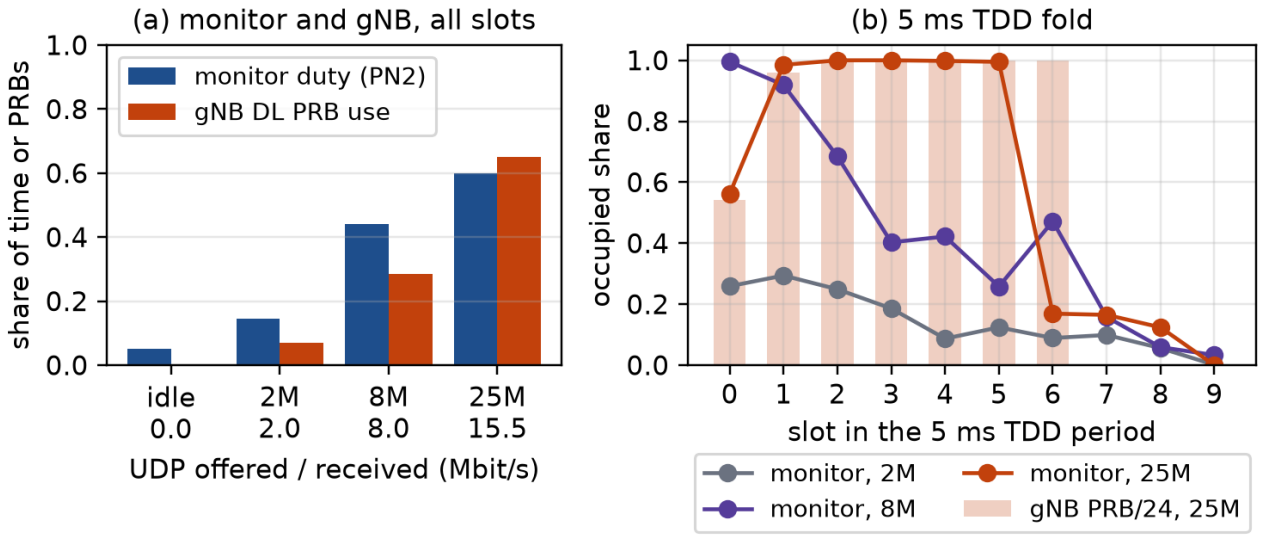


Figure 22: R2. (a) Duty cycle of the monitor on PN2 and DL PRB use of the gNB for four UDP loads. The labels give the offered and the received rates. (b) Occupied share of each 0.5-ms block in the 5-ms TDD period (monitor), and the gNB PRBs for each slot at 25 Mbit/s. The phase of the monitor fold is arbitrary, so the two slot axes are not aligned.

Limits. This is one run with 2-s records and three gNB messages for each load. The monitor has no slot timing, so we cannot align its fold with the slot index of the gNB. The 0.5-ms blocks do not start at the slot borders, which spreads the edges of the pattern. At 25 Mbit/s, the UE received 15.5 Mbit/s with 36 % loss, and the gNB MCS fell to 15 in one second. The PRB counts of the gNB are means over each 1-s message. The WebSocket interface is specific to OCUDU.

7.3. R3: when CHEM frequency-offset and CIR commands act

Why. The CHEM-controls rule of version 1 said that a frequency offset or a CIR acts after approximately 7 to 8 s, and a fixed SNR after 14 to 16 s. These values came from time stamps on the control computer. On 5 November 2026, the clock of the control computer was 13.3 s behind the clocks of FN1, PN1 and PN2 (13.29, 13.26 and 13.28 s). R1 showed that a fixed SNR acts within 0.6 s. R3 measures the frequency offset and the CIR with node clocks only.

Method. FN1 sent a 100-kHz tone. PN1 recorded 30 s with the CHEM poll (4 Hz). During the record, we sent four commands to the FN1 to PN1 link and time-stamped each on FN1. The commands were frequency offset +300 Hz, CIR echo2, CIR clear and frequency offset 0. `doppler_cfo.py` measured the tone frequency in 100-ms windows, and `r6_ana.py step` the power in 10-ms blocks. The tap set echo2 adds a −6 dB tap at 3 samples, which changes the tone level. After the timing record, FN1 sent five kinds of `siggen.py` again, and PN1 recorded 2 s of each for the spectra of Script 6 (Fig. 12).

Results. Fig. 23 shows the record.

- The frequency offset of +300 Hz acted 0.53 s after the time stamp of the command. The CIR echo2 acted after 0.50 s, and the CIR clear after 0.52 s. The CHEM link log showed each change after 0.73 to 0.75 s.
- With R1, all five measured commands (two fixed SNRs, one frequency offset and two CIRs) acted within 0.55 s. The time stamp is taken one SSH call before the command reaches CHEM, so the real delay is shorter.

- The tap set echo2 raised the tone level from −21.95 to −19.30 dBFS. A tap of −6 dB at 3 samples (1.5 μs) gives $20 \log_{10} |1 + 0.5\, e^{-j2\pi \cdot 100\,\mathrm{kHz} \cdot 1.5\,\mu\mathrm{s}}| = 2.64$ dB for a 100-kHz tone. CHEM therefore applies the taps without a normalization of the power.

- The values of version 1 (7 to 8 s for a frequency offset or a CIR, 14 to 16 s for a fixed SNR) are therefore not CHEM delays. They came from time stamps on the control computer. Logs with both clocks give the offset on the earlier days: approximately 7 s on 31 October, 8 s on 1 November and 9 s to 10 s on 3 November 2026. The clock of the control computer thus lost approximately 1.3 s each day. The 7 to 8 s values of 31 October and 1 November are this offset. The 14 to 16 s value for the fixed SNR came from one capture of 3 November without node time stamps (Report-6, concept 4). Its noise steps at 33 s and 69 s of the capture belong to the commands at 36 s and 72 s, not to the commands 14 s earlier. Table 3, Scripts 10 and 11 and the recommendations are corrected.

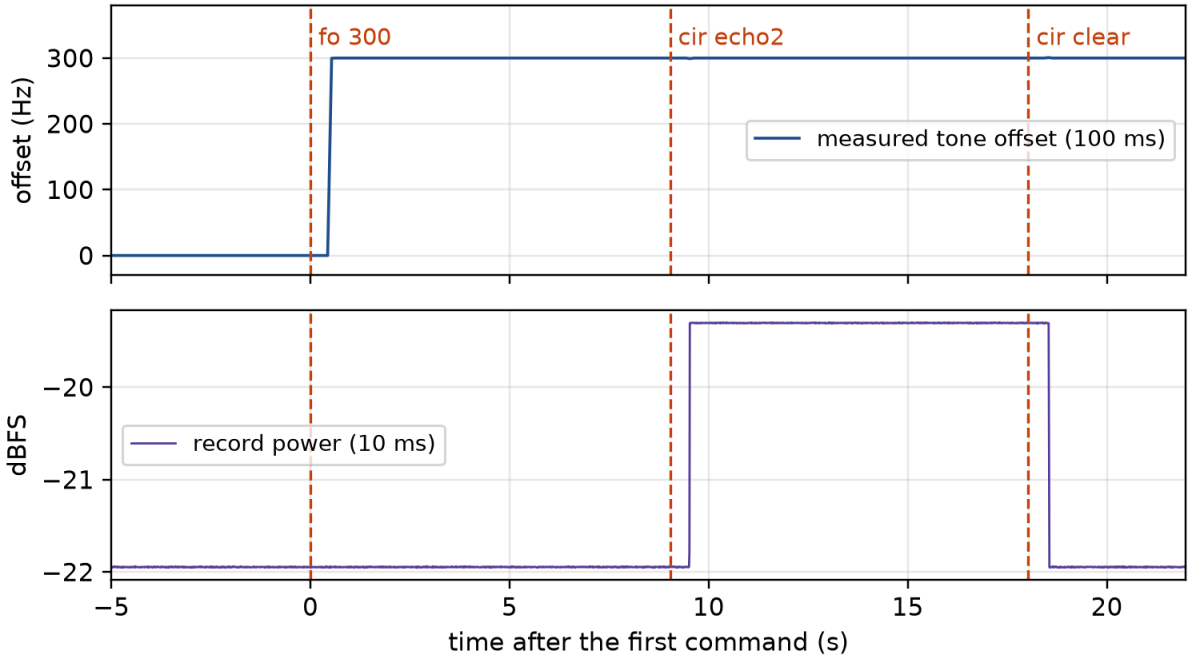


Figure 23: R3. Tone offset (top) and record power (bottom) at PN1 during three CHEM commands (dashed lines, time stamps on FN1). The fourth command came after the end of the record.

Limits. One run on one link. The last command (frequency offset 0) came after the end of the 30-s record, because the recorder started late, so it has no measurement. The clock offset of 13.3 s is the value of 5 November 2026. It can change, so time-stamp on the nodes. Report-2 measured 1.2 to 1.9 s from a fixed-SNR command to a change of the gNB PUSCH SNR. That delay includes the averaging of the gNB metrics.

7.4. N1: the scripts in the other allowed TX bands

Why. Rule 5 of Table 3 lists eight allowed TX bands. Version 2 of this report tested only 3400 MHz. We repeated the main steps in two other bands.

Method. The helper `vusrp_io.py` now holds the list of allowed bands. Its self-test on PN1 accepted 914 and 433.92 MHz and refused 1000 and 3500 MHz. FN1 sent a tone at 914 MHz and 2 MS/s with TX gain 45. PN1 ran `preflight.py rx` for 5 s. Then FN1 sent QPSK at −20 dBFS. On PN1, `chem_sweeper.py` set the CHEM fixed SNR to 20 and 10 dB and then to the link budget. At 433.92 MHz, FN1 sent a tone at 1 MS/s with TX gain 40. The sample band of 433.42 to 434.42 MHz stays inside the allowed range of 433.05 to 434.79 MHz.

Results. Table 6 gives the values.

- All seven preflight items passed in both bands. The tone was in the FFT bin at 100 006.1 Hz. The records had no overflow, no zero samples and no clip.
- The CHEM path loss fell from 77.43 dB at 3400 MHz to 66.47 dB at 914 MHz and 60.03 dB at 433.92 MHz.
- At 914 MHz, the data-aided SNR was 19.99 and 9.98 dB for the CHEM values 20 and 10 dB. At the link budget (13.62 dB), it was 13.61 dB. The error was 0.02 dB or less. This agrees with Script 11 at 3400 MHz.

Table 6: N1: preflight tone loop in three bands (single runs, FN1 to PN1, CHEM 2-ray, RX gain 40). SNR is the CHEM link SNR, and the level is the tone level at PN1.

Frequency (MHz)	Rate (MS/s)	TX gain	Path loss (dB)	SNR (dB)	Level (dBFS)	Preflight
3400 (Script 13)	2	60	77.43	25.3	−21.95	7 of 7
914	2	45	66.47	15.6	−31.86	7 of 7
433.92	1	40	60.03	21.3	−29.20	7 of 7

Limits. Each band had one run on one link with a tone and QPSK. We did not test the other five bands. The TX gains differ between the bands, so the levels do not compare path losses directly.

7.5. N2: late sample delivery seen by the report scripts

Why. The debug study of 6 and 7 November 2026 found RX samples that arrive 0.4 to 1.1 ms late (Finding 16). Version 2 of this report did not check its own records. We added an option to the recorder that stores the arrival lag.

Method. The option `RX.read(..., lag_log=True)` stores the wall time and the number of received samples after each read. The option `iq_recorder.py --lag-log` writes them to a `.npy` file. The arrival lag is the wall time minus the start time plus the count divided by the rate. The command `r6_ana.py lag` removes the median and counts the steps above 0.4 ms. FN1 sent a 100 kHz tone. PN1 recorded 120 s at 2 MS/s. After 20 s, it recorded 60 s at 15.36 MS/s. One read had 0.5 ms of samples. All times come from the node clock. Nothing else ran on PN1 during the records.

Results. Fig. 24 shows the lag.

- At 2 MS/s, the lag had 14 steps above 0.4 ms in 121 s. Seven steps were 0.4 to 1.1 ms, and seven steps were 3.9 to 8.3 ms. At 15.36 MS/s, the lag had 17 steps in 61 s. Ten steps were 0.4 to 1.1 ms, and seven steps were 4.3 to 13.6 ms.

- After a step, the lag fell back to zero within 1 ms of wall time, because the next reads returned at once. The study of 6 and 7 November found steps that lasted 0.1 to 1 s.
- The steps did not cluster in two phases of the second. Each 0.05-s bin held at most four step starts.
- Both records had no overflow, no timeout and no zero sample. The tone offset at 2 MS/s was 0.00 Hz in all 1200 windows of 100 ms. The sample values are therefore right.

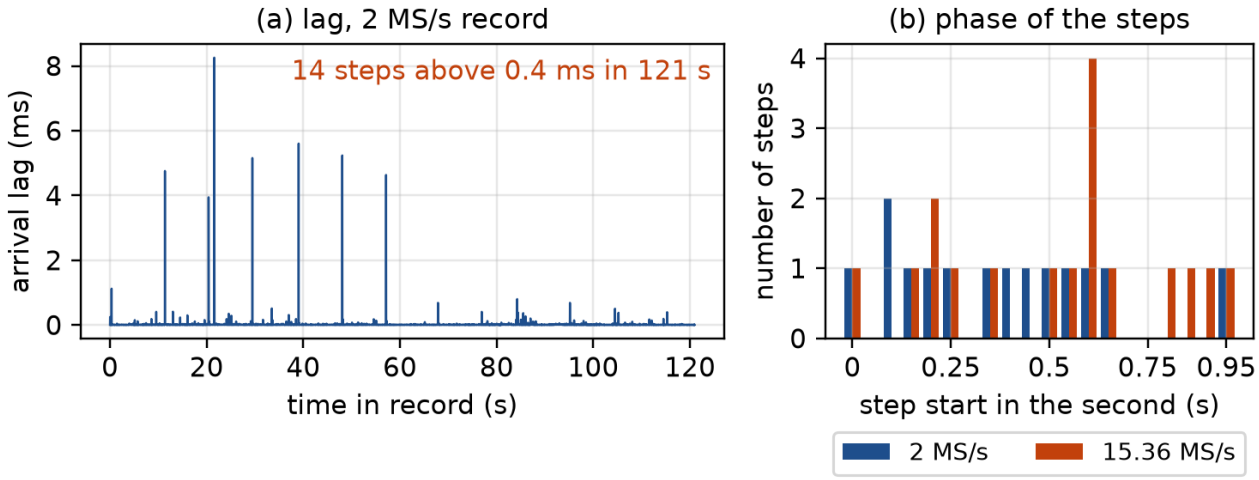


Figure 24: N2. (a) Arrival lag of the 2 MS/s record (maximum in each 30 ms). (b) Start of each step inside the second, for the 2 and 15.36 MS/s records.

Limits. The recorder sees only the time of each read. A late read can come from the twin or from a stop of the Python process. The test cannot separate the two causes. The lag here fell back faster than in the debug study. We did not run the probe of the debug study on the same nodes at the same time.

7.6. N4: the CHEM fixed SNR with a bursty signal

Why. Other measurements differ from the CHEM fixed SNR (Finding 5). CHEM sets the SNR from the TX power. We tested a TX with a duty cycle of 0.25.

Method. FN1 sent one noise burst of 1.2 MHz bandwidth. In each 10 ms, it sent 2.5 ms of noise and 7.5 ms of zeros. The mean level was −24 dBFS. On PN1, `chem_sweeper.py` set the CHEM fixed SNR to 20 and 10 dB and then to the link budget. Each record was 4 s long. Offline, `r6_ana.py bsnr` calculated the power in blocks of 0.5 ms. The on-blocks are the top 25 % of the blocks, and the off-blocks are the bottom 50 %. The first SNR is $(P_{\mathrm{on}} - P_{\mathrm{off}})/P_{\mathrm{off}}$ in the full band. The second SNR uses the mean power instead of P_{on}. As a control, FN1 sent continuous QPSK, and PN1 measured the data-aided SNR at 20 dB.

Results. Table 7 and Fig. 25 give the values.

- The noise level in the off-blocks followed the CHEM value: −45.04 dBFS at 20 dB and −35.04 dBFS at 10 dB. CHEM therefore scaled the noise with the TX power.
- The SNR during the bursts was 2.9 to 3.3 dB above the CHEM value. The SNR from the mean power was 2.7 to 2.9 dB below it.
- The control with continuous QPSK gave 19.99 dB for the CHEM value 20 dB.

Table 7: N4: CHEM fixed SNR and measured SNR for a bursty TX with a duty of 0.25 (single run, FN1 to PN1, full 2 MHz band).

CHEM value (dB)	P_{on} (dBFS)	P_{off} (dBFS)	SNR in bursts (dB)	SNR from mean power (dB)
20	−22.14	−45.04	22.88	17.08
10	−21.73	−35.04	13.11	7.10
Link budget (22.48)	−21.93	−47.70	25.76	19.74

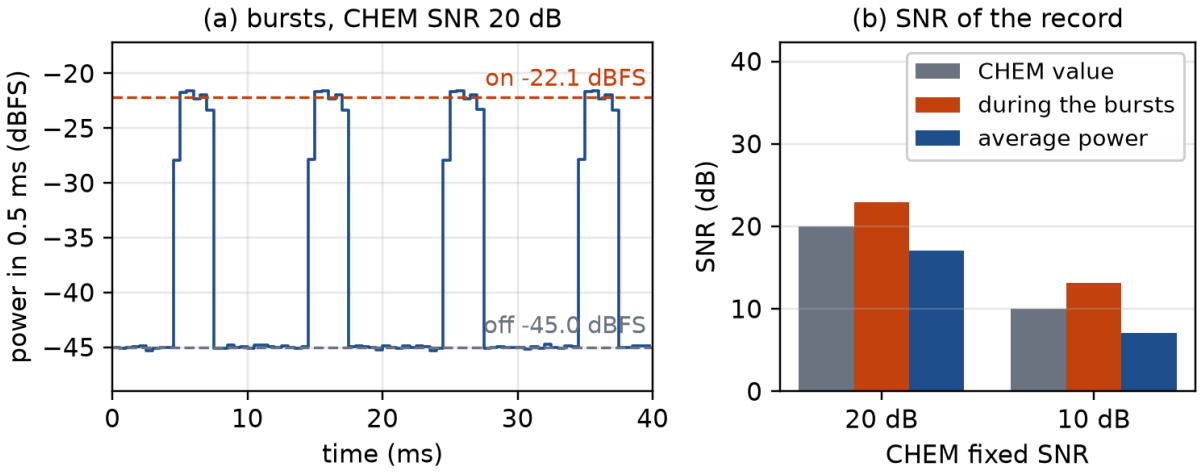


Figure 25: N4. (a) Power in blocks of 0.5 ms at the CHEM value of 20 dB, with the on level and the off level. (b) CHEM value, SNR during the bursts and SNR from the mean power.

Limits. We tested one duty cycle and one waveform. The block edges are not aligned with the bursts, which can bias the on level by 0.1 dB. We do not know how CHEM measures the TX power. The CHEM noise matched a power between the burst power and the mean power. Use the CHEM value as a mean-power label, and calculate your own SNR for bursty signals.

7.7. N3: the noise floor against the sample rate on one node

Why. R2 compared two nodes at two rates. N3 uses PN1 only and also gives a new level figure for Script 1.

Method. PN1 recorded 4 s with no TX at 2 MS/s. Then FN1 sent a TX of zeros at 1, 2, 7.68 and 15.36 MS/s, and PN1 recorded 4 s at each rate. Last, FN1 sent a 100 kHz tone at 2 MS/s. All records used 3400 MHz and RX gain 40.

Results. Table 8 gives the levels. The floor with a TX of zeros followed $-47.6 + 10\log_{10}(f_s/2\ \text{MS/s})$ within 0.05 dB. CHEM therefore adds noise with a constant density. The level with no TX was −80.98 dBFS, and the tone level was −21.95 dBFS. All records had no overflow.

Limits. One link, one gain and one record for each rate. The tone and the TX of zeros had the same TX settings as in Script 1.

Table 8: N3: median 1-ms power at PN1 (single records of 4 to 5 s).

Record	Level (dBFS)	Formula (dBFS)
No TX, 2 MS/s	−80.98	–
TX of zeros, 1 MS/s	−50.65	−50.61
TX of zeros, 2 MS/s	−47.60	−47.60
TX of zeros, 7.68 MS/s	−41.78	−41.78
TX of zeros, 15.36 MS/s	−38.75	−38.71
Tone, 2 MS/s	−21.95	–

8. Public GitHub Repository Catalog

This section gives the public Python SDR repositories from our research report and notes (`reports/` and `research_notes/` in our working folder). The search used the GitHub API on 4 November 2026. **We did not run these repositories in the twin.** We read only their source code and their documentation. Therefore, each verdict is a prediction from the rules of Section 4, not a test result. Treat a repository as confirmed only after it passes `preflight.py` and a test against a TX of zeros at 3400 MHz.

Verdicts.

- Direct use: only configuration changes, for example the frequency, the rate or the duration. Libraries for I/Q files are also in this class, because they do not use the radio.
- Small changes: a small code change, usually 5 to 10 lines. The most frequent change replaces the finite `num_done` burst with `start_cont`, a read loop and `stop_cont`.
- Not suitable: the repository needs hardware, a driver or a signal that the twin does not have, or it has no license. Its digital signal processing (DSP) can still be useful as a model.

The column "Link to this report" gives the script of Section 6 that uses the same method, or that replaces the repository in the twin. The UHD 4.9 helpers are important for a port to the twin. In UHD v4.9.0.0, `MultiUSRP.recv_num_samps()` uses `start_cont`, reads N samples, uses `stop_cont` and flushes. `send_waveform()` sets the end of burst only once, after the full loop. Therefore, code on these helpers follows the continuous-stream rule if each run makes one long call. However, `recv_num_samps()` sets the rate, the frequency and the gain again in each call. A call in a loop over LOs is therefore a forbidden retune.

8.1. UHD Python examples and textbook code

The official Ettus examples and the PySDR textbook give the radio I/O pattern for all other scripts.

Table 9: UHD Python examples and textbook code (not run on the twin).

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/EttusResearch/uhd/tree/master/host/examples/python (v4.9.0.1): `benchmark_rate.py`, `rx_spectrum_to_asciiplot.py`	Direct use	Both use `start_cont`. Set the frequency to 3400 MHz and the rate to 1 to 3 MS/s.	`preflight.py`
Same repository: `rx_to_file.py`, `rx_spectrum_to_pyplot.py`, `rx_to_remote_udp.py`	Small changes	Replace `num_done` with `start_cont` (`rx_to_file.py` line 166, `rx_spectrum_to_pyplot.py` line 194). Use 30 s or more, not the 0.5 s and 5 s defaults. Use absolute gains.	`iq_recorder.py`
Same repository: `tx_waveforms.py`	Small changes	Use the `--duration 0` path (`start_cont`) or change line 272. Use a TX in an allowed band of rule 5 (for example 3400 MHz).	`siggen.py`
Same repository: `usrp_power_meter.py`, RFNoC and X440 examples	Not suitable	Calibration data, RFNoC and X440 hardware are not in the twin.	–

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/777arc/PySDR	Direct use / Small changes	USRP snippets use `recv_num_samps` and a `start_cont` loop. Change the frequency and rate only. The DSP chapters (PSD, sync, OFDM, detection) operate without change.	`doppler_cfo.py`, `siggen.py`

8.2. Recorders and spectrum tools on the UHD Python API

These tools record or display spectra through `import uhd`. A sweep shows only the active link in the twin. Therefore, fix each tool to one LO at the TX frequency.

Table 10: Recorders and spectrum tools on the UHD Python API (not run on the twin).

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/genesys-neu/ni-rf-data-recording-api (MIT)	Small changes	Automated SigMF data-set recorder on `recv_num_samps`. Set the internal clock reference, one device and a rate in range.	`iq_recorder.py`
https://github.com/metadavi/rfnode (GPL-3.0)	Direct use / Small changes	`capture_uhd.py` and `peakhold_uhd.py` use `start_cont`. Change `num_done` in `survey_uhd.py` line 98.	`iq_recorder.py`
https://github.com/Andreas237/usrp_python (Unlicense)	Direct use	70 MHz to 6 GHz sweep with Welch PSD. Fix it to one LO at the TX frequency.	`occupancy_monitor.py`
https://github.com/ssk1504/SDR_USRP_spectrum_analysis (MIT)	Small changes	B210 short-time Fourier transform (STFT) capture. Change `num_done` in `capture_stft.py`.	`occupancy_monitor.py`
https://github.com/agile-radio-lab/fft-remote	Direct use / Small changes	Spectrum analyzer without GNU Radio, with `start_cont`. The code is from 2020 and has no license.	`occupancy_monitor.py`
https://github.com/seitzbg/sdr-tools (MIT)	Small changes / Not suitable	Its sweep tunes and then sends a `num_done` burst for each step. This breaks two rules. Use one fixed tune and `start_cont`.	`fixed_tune_scanner.py`
https://github.com/wineslab/uhd-mcp (MIT)	Small changes	MCP server for USRP discovery, signal generation and capture. Add the twin TX band to its safety limits. Check its capture path for `num_done`.	Section 3
https://github.com/NTIA/scos-usrp (GPL-3.0)	Not suitable	The GPSDO clock and time source are hard-coded. It needs the full scos-sensor (Django) stack.	`occupancy_monitor.py`

8.3. Channel sounders and link modems

Table 11: Channel sounders and link modems (not run on the twin).

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/anilgurses/USRP-Channel-Sounder (no license file)	Small changes	Zadoff-Chu (ZC), PN, chirp and OFDM probes with SigMF output. Build the Cython file `ChsRX.pyx` again for Python 3.10 (it has a cpython-311 `.so`). Use the internal clock and time, not `gpsdo`. TX in an allowed band of rule 5 (for example 3400 MHz), same frequency and rate on both nodes.	`channel_sounder.py`

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/minrhmn/ofdm-ota-cosmos (MIT)	Small changes	numpy OFDM frame generator, TX, RX and SNR/bit error rate (BER) sweep. Change `num_done` in `rx_capture_ota.py`, use `--ref internal`, and change the X310 defaults to B205.	`siggen.py ofdm`, `meters.py`
https://github.com/dchutchings/py_ofdm (GPL-3.0)	Direct use	Offline OFDM receiver for SNR, EVM and PAPR per subcarrier. It reads I/Q arrays only.	`meters.py`

8.4. GNU Radio based AERPAW labs and recorders

These repositories use the GNU Radio `gr-uhd` blocks, not `import uhd`. The twin image has GNU Radio 3.10.12. GNU Radio OFDM failed at 5.76 MS/s or more in the twin, so keep these labs at 1 to 2 MS/s. Report-1 gives more GNU Radio code.

Table 12: GNU Radio based AERPAW labs and recorders (not run on the twin).

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/teaching-on-testbeds/rf-fingerprint (MIT)	Direct use	Written for AERPAW: 3.32 GHz, 1 MS/s.	Report-1
https://github.com/teaching-on-testbeds/ofdm, https://github.com/teaching-on-testbeds/digital-modulation (MIT)	Small changes	Written for COSMOS. Set the frequency to an allowed band of rule 5 and the rate to 1 to 2 MS/s.	`siggen.py`
https://github.com/teaching-on-testbeds/uav-wireless-localization (MIT)	Direct use / Small changes	Received signal strength indicator (RSSI) localization notebooks for AERPAW UAV data. We did not read its radio code.	`iq_recorder.py` (CHEM log)
https://github.com/spectregrams/spectre (GPL-3.0)	Small changes	Mature recorder and spectrogram tool. It needs a GNU Radio 3.10 or Docker build in the twin and one USRP channel.	`iq_recorder.py`

8.5. SigMF input and output, viewers and data sets

These tools read I/Q files only. They operate on central processing unit (CPU) without radio changes.

Table 13: SigMF tools, viewers and data sets (not run on the twin, except sigmf-python).

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/sigmf/sigmf-python (LGPL-3.0)	Direct use	The I/O layer: `sigmf.fromarray`, `add_capture`, `tofile`, `fromfile`. Our scripts use version 1.14.0 as a private install.	Scripts 1, 2, 14
https://github.com/sigmf/SigMF (CC-BY-SA-4.0)	Direct use	The SigMF specification. The core fields and the `antenna` extension give most UAV metadata. No extension gives the CHEM link state.	`aerpaw:` namespace
https://github.com/IQEngine/IQEngine (MIT)	Direct use	Web viewer for SigMF. The plugins `simple_detector`, `markos_detector` and `template_plugin` write detector boxes back as annotations. Use it off the node.	`occupancy_monitor.py`

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/miek/inspectrum (GPL-3.0)	Direct use	Desktop viewer for cf32 and SigMF files (C++).	`iq_recorder.py`
https://github.com/daniestevez/sigmf-toolkit (MIT), https://github.com/emrefbulut/iqforge (MIT)	Direct use	Small SigMF utilities, and a tool that makes PyTorch data sets from SigMF. Both are new and small.	`chem_sweeper.py`
AERPAW spectrum data set, Feb 2022, https://datadryad.org/dataset/doi:10.5061/dryad.hmgqnk9zn	Direct use	SigMF records with a Python reader.	`mat2sigmf.py`

8.6. Measurement DSP libraries

These libraries give modems, filters and sync loops for EVM and SNR on single-carrier test waveforms. They read numpy arrays only.

Table 14: Measurement DSP libraries (not run on the twin).

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/mwickert/scikit-dsp-comm (BSD-2)	Direct use	Phase-shift keying (PSK) and quadrature amplitude modulation (QAM) modems, filters, sync loops.	`meters.py`
https://github.com/rwnobrega/komm (GPL-3.0)	Direct use	Analog and digital modems and codes for reference values of BER and EVM.	`meters.py`
https://github.com/mhostetter/sdr (MIT, not confirmed)	Direct use	Sync and measurement functions with Numba.	`meters.py`, `doppler_cfo.py`
https://github.com/veeresht/CommPy (BSD-3)	Direct use	Modems, channel codes, channel models. The maintenance slowed after 2023.	`siggen.py`

8.7. Spectrum occupancy and energy detection

No public repository is a direct occupancy monitor for `import uhd`. These repositories are models for the algorithms. The NTIA format is also the format of the NIST citizens broadband radio service (CBRS) SEA data sets at 3530 to 3710 MHz.

Table 15: Spectrum occupancy tools (not run on the twin).

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/NTIA/scos-actions	Small changes	Single-frequency FFT statistics, stepped-frequency I/Q, Y-factor calibration and a `SigMFBuilder`. Its actions call an abstract signal analyzer, so a small UHD adapter is necessary. Its stepped-frequency action breaks the rule against a retune.	`occupancy_monitor.py`, `fixed_tune_scanner.py`
https://github.com/NTIA/sigmf-ns-ntia	Direct use	SigMF namespace for sensor, environment and algorithm fields.	`iq_recorder.py`
https://github.com/usnistgov/nasctn-sea-ingest	Direct use	Reader for the NIST SEA data. A model to organize long occupancy data sets.	`occupancy_monitor.py`

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/mac3d0/rf-occupancy-monitor (MIT)	Not suitable (model)	Median noise floor and duty-cycle classes from `rtl_power` bins. Replace the input with a UHD Welch PSD.	`occupancy_monitor.py`
https://github.com/TheBengineer/SDRwatch (MIT)	Not suitable (model)	Constant false alarm rate (CFAR) with a median and median absolute deviation (MAD) noise floor. The project is new (0 stars).	`occupancy_monitor.py`
https://github.com/vineeths96/Spectrum-Sensing-for-Cognitive-Radio (MIT)	Not suitable (model)	Energy and cyclostationary detectors from 2020.	`occupancy_monitor.py`
https://github.com/gnuradio/gr-inspector (GPL-3.0)	Small changes	Signal detection and OFDM parameter estimation in GNU Radio. Use a file source with our records.	`dl_capture.py`

8.8. Classification and AERPAW analysis code

The twin knows each waveform (srsRAN, OCUDU, OAI, GNU Radio and `siggen.py`). Therefore, each record has an exact label. Train a classifier on synthetic data at the twin rate, then check it on twin records.

Table 16: Classification and AERPAW analysis code (not run on the twin).

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/TorchDSP/torchsig (MIT)	Small changes	60 or more signal types, Python 3.10 or later. CPU with 4 or more cores is supported, a graphics processing unit (GPU) is recommended. The README does not tell about pretrained models or SigMF input.	`chem_sweeper.py`, `iq_recorder.py`
https://github.com/brysef/rfml (BSD-3)	Direct use	Baseline convolutional neural network (CNN) models for modulation classification.	`siggen.py` (labels)
https://github.com/Richardzhangxx/AMR-Benchmark	Not suitable	No license, so reuse is restricted.	–
https://github.com/radioML/dataset	Not suitable	Old (2017) GNU Radio data set generator, replaced by TorchSig.	–
https://github.com/NVlabs/sionna	Direct use	NR link-level and channel models for a comparison with twin records. CPU is slower.	`meters.py`
https://github.com/AERPAW/AERPAW-DT-SIONNA-EXTENSION	Not suitable (unknown)	AERPAW digital-twin extension of Sionna. It has no description and no license yet.	–

8.9. LTE and NR decoders

No mature pure-Python SIB1 decoder exists. In the twin, the Rust nr-sib1 decoder operated on an OCUDU record (Script 8). The SSB of 240 subcarriers at 30 kHz spans 7.2 MHz, so 7.68 MS/s is sufficient for PCI and MIB. Use a 10 MHz cell at 15.36 MS/s and a 24-RB control resource set (CORESET)#0 in the twin. LTE PCI detection needs only the center 1.08 MHz.

Table 17: LTE and NR decoders (not run on the twin, except nr-sib1).

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/zackiles/nr-sib1 (MIT)	Direct use	Rust decoder: PSS/secondary synchronization signal (SSS), physical broadcast channel (PBCH), Type0-PDCCH, downlink control information (DCI), PDSCH low-density parity-check (LDPC) and ASN.1. No Python bindings, so call it with `subprocess`. Its README lists 30.72 and 61.44 MS/s, but it operated at 15.36 MS/s in the twin.	`dl_capture.py` (tested)
https://github.com/catkira/py3gpp (AGPL-3.0)	Small changes	Python functions in the form of the MATLAB 5G Toolbox (`nrPSS`, `nrPBCH`, `nrPDSCHDecode`, `nrLDPCDecode`). It has no PDCCH or DCI module, so SIB1 needs a DCI 1_0 step of your own.	`dl_capture.py`
https://github.com/danielstevez/jupyter_notebooks (5G NR downlink notebook)	Small changes	PBCH and MIB on an srsRAN gNB SigMF record at 15 kHz. For n78, change to 30 kHz and SSB Case C.	`dl_capture.py`
https://github.com/spritelab/5GSniffer	Small changes	C++ PDCCH blind decoder on srsRAN, with file input. Use it only on your own gNB.	`dl_capture.py`
https://github.com/JiaoXianjun/LTE-Cell-Scanner (AGPL-3.0), https://github.com/srsran/srsRAN_4G physical layer (PHY) examples (AGPL-3.0)	Small changes	C tools for LTE cell identifier (ID), MIB and system information block (SIB) from cf32 files. Use 11.52 to 15.36 MS/s for a 10 MHz cell.	`dl_capture.py`
https://github.com/falkenber9/falcon, https://github.com/SysSec-KAIST/LTESniffer (AGPL-3.0)	Small changes	DCI-level load of an LTE cell. Use them only on your own eNB.	`occupancy_monitor.py`
https://github.com/GigayasaWireless/5G-on-SDRs	Not suitable	Tutorials on Pluto. The license of the library is not clear.	–
https://github.com/mdisterhof/nrphypy (GPL-3.0)	Not suitable	Not maintained after 2022, and the scope is not clear.	–
https://github.com/rockyco/5G-Scanner	Not suitable	Needs an X310 field-programmable gate array (FPGA) image. NR cells do run in the twin (OCUDU, OAI), but the hardware is not there.	`dl_capture.py`

8.10. SoapySDR, Pluto, RTL-SDR and other hardware

These tools reach the V-USRP only through SoapyUHD, or not at all. The Ubuntu 22.04 package `soapysdr0.8-module-uhd` depends on `libuhd4.1.0`. An apt UHD package breaks the UHD 4.9 fork of the nodes. Therefore, SoapyUHD needs a build against the node UHD, and nobody has tested its burst behavior. It is easier to copy the DSP of these tools into an `import uhd` script. Replace `sdr.rx()` or `read_samples()` with a `start_cont` loop, and scale the samples to fc32.

Table 18: SoapySDR, Pluto, RTL-SDR and other hardware (not run on the twin).

Repository	Verdict	Changes for the twin	Link to this report
https://github.com/pothosware/SoapySDR, https://github.com/pothosware/SoapyUHD	Small changes	Build SoapyUHD from source against the UHD 4.9 fork. Do not install the apt package.	–
https://github.com/xmikos/soapy_power, https://github.com/pierreay/soapyrx	Small changes	Need SoapyUHD. soapy_power is from 2017. Check soapyrx for finite bursts.	`occupancy_monitor.py`
https://github.com/rafafunt/FalconSDR, https://github.com/samarth1011-1/Cross-SDR-Spectrum-Analyzer	Not suitable (model)	Web or Qt graphical user interface (GUI) tools. Their squelch, energy detection, occupied bandwidth (OBW) and hold functions are useful as models.	`occupancy_monitor.py`
https://github.com/analogdevicesinc/pyadi-iio, https://github.com/pyrtlsdr/pyrtlsdr, https://github.com/GvozdevLeonid/python_hackrf	Not suitable	Pluto, RTL-SDR and HackRF drivers. Copy only the DSP.	–
https://github.com/subeep/deye	Not suitable	Dual-channel C++ recorder. The V-USRP has one channel.	–
https://github.com/jopohl/urh, https://github.com/ainfosec/FISSURE	Not suitable	Large GUI frameworks. URH is archived.	–
https://github.com/junzis/pyModeS, https://github.com/M0r13n/pyais, Wi-Fi, automatic dependent surveillance-broadcast (ADS-B) and global navigation satellite system (GNSS) tools	Not suitable	The V-USRP receives only other AERPAW nodes through CHEM. Broadcast signals are not there.	–

9. Findings and Recommendations

9.1. New findings from the script tests

The tests of this report confirm the rules of Section 4 and give these new findings about the twin. Each finding gives the script or the experiment (R1 to R3) that showed it.

1. **CHEM settings act only on links that exist.** A CIR command with no RX stream acted on no link: CHEM returned an empty list, and the PDP showed one tap (Script 5). Keep the RX and TX streams open when you change CHEM, as `chem_sweeper.py` does (Script 11).

2. **CHEM commands act within approximately 0.5 s.** A fixed SNR, a frequency offset and a CIR acted 0.49 to 0.55 s after the time stamp of the command on FN1 (R1, R3). The longer values of version 1 came from times on the control computer, whose clock was approximately 7 s (31 October) to 13.3 s (5 November) behind the nodes (Section 7.3). Time-stamp on the nodes.

3. **`/root/reset.sh` stops CHEM for approximately 12 s.** During the reset, both CHEM ports stopped for 12.0 s. After a stop, `reset_chem` took 15.9 s, and the ports answered again 26.9 s after the stop (Script 12). A watchdog must use a grace time above the reset time.

4. **A loaded LTE cell fills the clip level.** During an iperf3 DL at 35.1 Mbit/s, 35 % of the LTE samples at PN2 were at the −16 dBFS clip. In the idle LTE record, the value was 4.9 % (Scripts 3 and 8). The measured load power is therefore a lower limit. Use a lower TX gain or a monitor node with more path loss.

5. **The CHEM fixed SNR is exact for a continuous signal over the full sample band.** The data-aided SNR agreed with the CHEM value to 0.01 dB from 0 to 30 dB (Scripts 7 and 11). The SNR from the EVM was 6.02 dB higher at 4 samples per symbol. CHEM keeps the signal power and adds noise. A settle time of 3 s gave the same accuracy (0.04 dB) and made a sweep 3.5 times faster (R1). Other measurements differ from the CHEM value. The gNB PUSCH SINR was 3 to 4 dB off (debug report, L3). An OFDM receiver measured 2 to 8 dB less (Report-6, concept 4). The CHEM SNR is an average-power SNR over the full sample band, and it changes with the TX load (Report-2 Part-2). A receiver SNR after a filter, or during the active part of a burst, is therefore different. With a burst duty of 0.25, the SNR during the bursts was 2.9 to 3.3 dB above the CHEM value (N4). The twin is a calibrated and fast source of labeled data for SNR estimators, when you use the CHEM definition.

6. **The twin adds no CFO, and the CHEM offset is exact.** With a CHEM offset of 0, the tone had no offset. The offsets +300 and −200 Hz were exact to 0.001 Hz, and each step came within 0.2 s of the CHEM log (Script 10).

7. **The CHEM CIR taps are exact in samples.** The measured PDP agreed with the pychem tap tables to 0.01 dB (Script 5). The tap set echo2 raised a 100-kHz tone by 2.65 dB, as the taps predict (2.64 dB), so CHEM does not normalize the tap power (R3).

8. **The noise floor depends on a registered TX.** At PN1 and 2 MS/s, the level was −81.0 dBFS with no TX and −47.6 dBFS with a TX of zeros (Script 1). A noise-reference SNR without a registered TX is 33.4 dB too high (Script 7). The CHEM noise has a constant density: on one node, the floor at 1, 7.68 and 15.36 MS/s followed the ratio of the rates to 0.05 dB (N3). R2 gave the same ratio on two nodes.

9. **A retune scanner aliases periodic traffic.** A virtual stepped scan with a revisit equal to the burst period gave 1.00, 1.00, 0.00 and 0.00 for the duty cycles 1, 0.5, 0.25 and 0 (Script 4). A fixed-tune channelizer gave the correct values.

10. **An energy monitor sees the TDD pattern, not the PRB load.** Next to a live OCUDU cell, the duty cycle of `occupancy_monitor.py` was 1.5 to 2 times the PRB use of the gNB at low load (0.14 against 0.07). At full buffer, it stopped at the DL share of the TDD pattern (0.60 against 0.65). The gNB WebSocket metrics give the labels that the monitor cannot see (R2).

11. **The off-channel UHD2 level is noise.** At the 3410 MHz LO, the UHD2 snapshots had a flat spectrum at −51.9 dBFS (Script 14). This level is below the CHEM noise at 3400 MHz (−47.6 dBFS). We do not know the cause.

12. **The NR decoder reports a CFO that CHEM does not apply.** nr-sib1 reported −3.5 kHz on the OCUDU record, but CHEM reported an offset of 0 (Script 8). The SIB1 decode operated in 16 of 26 windows of 100 ms.

13. **TCP over the OCUDU link has long queues.** The median ping RTT went from 24.6 ms to 1114 ms during a TCP DL of 4.59 Mbit/s, and 7 of 25 intervals had zero throughput (Script 9).

14. **The V-USRP reports a master clock rate of 0.001 Hz.** A script must not calculate a rate from this value (Script 13).

15. **The continuous stream is reliable at the tested rates.** All records at 2, 11.52 and 15.36 MS/s had no overflow, no timeout and no zero samples, and CHEM did not stop outside the planned resets (Script 1). But the arrival time of the samples is not constant (next finding).

16. **The twin can deliver samples about 1 ms late, in steps.** A receive loop with time stamps on a node (listing below) showed steps of +0.4 ms to +1.1 ms in the arrival lag. A step lasts 0.1 s to 1 s and then decays. Several nodes see the same step, and the steps follow a 2 Hz pattern in the wall-clock time. The sample values are right, no overflow is reported, and in a pulse test no sample was missing. An I/Q record therefore looks normal. A real-time receiver can fail: the OAI 5G UE sends no uplink during a step, and the gNB can drop it (Report-2 and Report-2 Part-2). The debug study of 6 and 7 November 2026 found this. The recorder of this report also sees late reads: 14 steps above 0.4 ms in 121 s at 2 MS/s and 17 in 61 s at 15.36 MS/s (N2). In this test, the lag fell back within 1 ms, not over 0.1 to 1 s. We cannot say if the twin or the Python process caused them. The process in the twin that makes the steps is not known.

Detect late samples. The loop reads the continuous stream and stores the wall time after each read and the number of samples received so far. The expected time of the last sample is the start time plus the count divided by the rate. The difference is the arrival lag. A step of more than 0.4 ms is a late delivery. Use the clock of the node for both times, not the clock of the control computer. The probe `rx_lat.py` of the debug study does this with a plain UHD stream, without any processing load.

```
st.issue_stream_cmd(cmd)                # start_cont, stream_now = True
n_tot, lag, t0 = 0, [], None
while time.time() < t_end:
    k = st.recv(buf, md, 1.0)
    wall = time.time()                  # right after recv returns
    n_tot += k
    if t0 is None: t0 = wall - n_tot / rate
    lag.append(wall - (t0 + n_tot / rate))   # arrival lag of the last sample
lag = np.array(lag) - np.median(lag)     # the constant part cancels
print("steps > 0.4 ms:", int(np.sum(np.diff(lag) > 4e-4)))
```

9.2. Limits of this report

- All nodes stayed on the ground, so Script 10 did not see a Doppler shift from a flight. CHEM applies one Doppler shift to all taps of a link, so a flight adds a frequency offset only. Report-3 measured this shift in flights with its own receivers.
- Most results come from one link (FN1 to PN1, 36.7 m) with one channel model (2-ray), at 2 MS/s, and from one run. We give no spread over repeated runs, except for the sweeps.
- The timing and the levels are for CHEM 1.2.4 and the software of Table 19. A new CHEM version can change them.
- The public repositories of Section 8 were not run. Their verdicts are predictions.
- The occupancy results use a simple energy detector with a fixed threshold. The cell results come from one LTE and one NR configuration.
- We measured the late samples with the recorder of this report in one test (N2). The recorder cannot tell the twin from a stop of the Python process. The debug study used its own probe.
- The causes of three effects are not known: the UHD1 stop of CHEM, the −3.5 kHz CFO of the NR decoder, and the low off-channel level of UHD2.

9.3. Recommendations

1. Start each campaign with `preflight.py`. Then start the TX before the RX, and keep both streams open during all CHEM changes.
2. Use `vusrp_io.py` in all new Python radio code. Use it also for a port of a public UHD script, to replace `num_done` and to stop a retune.
3. Use 1 to 3 MS/s for link tests and 11.52 to 15.36 MS/s for cell records. Do not use 23.04 MS/s or more for regular work.
4. Keep the RX level below −18 dBFS. Read the clip fraction from `meters.py` before you use a power value.
5. Record each test with `iq_recorder.py`. The CHEM link log in the SigMF file makes each record a labeled data set for channel, ISAC and classifier work. For ISAC work, use only the relative tap delays and the Doppler shift. CHEM adds no propagation delay.
6. Use `chem_sweeper.py` for SNR, CFO and CIR studies, with a settle time of 3 s. Label the records with `ds_manifest.py` (Section 7.1). Time-stamp on the nodes, not on the control computer.
7. Next to an OCUDU cell, log the WebSocket metrics (port 8001) with each spectrum record. Use the PRB use of the gNB as the label of the record (Section 7.2).
8. Do not run the UHD1 sample in the twin. Use `occupancy_monitor.py` or `fixed_tune_scanner.py` at the TX frequency. Use UHD2 only with the LO at the TX frequency, and convert its output with `mat2sigmf.py`.
9. Run `chem_watchdog.py` with a grace time of 60 s or more during long campaigns, because CHEM is shared by all users.

10. Before you use a public repository of Section 8, run it once against a TX of zeros at 3400 MHz. Treat the verdict as confirmed only after this test.

11. For a real-time receiver (a 5G UE or a packet decoder), run the time-stamped receive loop above before the main test. If the lag has steps of 0.4 ms to 1.1 ms, expect short outages. Add a tolerance in the receiver, for example the gNB setting of Report-2 that stops the radio link failures (RLFs) from short UL pauses.

12. Do these tests next:

    - Script 10 during a drone flight, to compare the measured shift with the CHEM Doppler.
    - The 3-s settle time in CIR and frequency-offset sweeps.
    - The cause of the -3.5 kHz decoder CFO.
    - The off-channel level at 3410 MHz.
    - The UHD1 CHEM stop with a read size equal to `max_num_samps`.

10. Reproduce It

This section gives the software versions and the full command sequences of our sessions. With them, a team can repeat each result of this report in its own experiment. The session scripts `sess.sh` (sessions S1 to S4b, 4 November 2026) and `sess_r6.sh`, `sess_r6b.sh` (sessions R1 to R3, 5 November 2026) in `gnuradio_deploy/work/deployX/` contain the same commands. The script `sess_r5py.sh` in `results/R5py_nov10/` holds the sessions N1 to N4 of 10 November 2026. They also take the radio lock and copy the logs.

10.1. Software versions

Table 19 gives the versions on the nodes. We read them on 5 November 2026. They were the same on 4 November 2026.

Table 19: Software on the nodes of experiment X1532, as of 5 November 2026.

Item	Version and use
Operating system	Ubuntu 22.04.5 LTS, kernel 5.15 (node containers)
Python and libraries	Python 3.10.12, numpy 2.2.0, scipy 1.14.1, matplotlib 3.10.0 (system packages of the image)
UHD and V-USRP	UHD 4.9.0.HEAD-aerpaw (Python API `import uhd`), V-USRP of the B205 type
SigMF	sigmf-python 1.14.0, private install (`pip install -target _vendor -no-deps sigmf==1.14.0`), system jsonschema 3.2
CHEM	version 1.2.4, control port 5000, V-USRP configuration port 6000
GNU Radio	3.10.12.0 (only for the comparison with Report-1)
LTE stack	srsRAN 4G (`srsenb`, `srsepc`, `srsue`), binary version 25.10.0, SE1 sample scripts of the image
5G gNB and core	OCUDU gNB commit `baeeb4b477`, Open5GS v2.6.4
5G UE	OAI `nr-uesoftmodem`, commit `01ef2b3` (24 August 2026)
NR decoder	nr-sib1 (Rust), built in an earlier session (deployL), called with `subprocess`
Traffic	iperf 3.9, ping (iputils)
Our scripts	`node_tools/aerpaw_py/` of this report (14 scripts, `ds_manifest.py` and `r6_ana.py`), `node_tools/chemctl.py`, `node_tools/oran/ws_tool.py`

10.2. Session frame

Each radio session of this report had the same frame:

1. Take the radio lock on FN1: `mkdir /root/claude_radio_lock` succeeds only for one owner.
2. Stop the old radio programs on the nodes FN1, FN2, PN1 and PN2 and wait 20 s.
3. Reset the experiment (`/root/reset.sh` on FN1) and wait until the CHEM ports 5000 and 6000 answer (55 to 60 s).
4. Set the channel: `chemctl.py set 2_RAY -0.3 -shadow 0`.
5. Run the steps of the session. Start each TX 10 s before its RX. Wait 20 s after each stop.
6. Copy the logs. Delete the large `.sigmf-data` files on the node. Check the CHEM ports. Release the lock.

10.3. Commands for the script tests

All commands run in the script folder on the node, after `source /root/.ap-set-experiment-env.sh`. TX on FN1 and RX on PN1 at 3400 MHz and 2 MS/s, unless the line tells otherwise. `CC` stands for `python3 /root/claude_tools/chemctl.py`. Start each RX approximately 10 s after its TX.

```
# Scripts 1, 13: rules, self-test and noise levels
python3 vusrp_io.py                                  # rule self-test, no radio
python3 preflight.py check                           # on FN1 and on PN1
python3 preflight.py tx --secs 40                    # FN1
python3 preflight.py rx --secs 5 --out pf_rx         # PN1
python3 iq_recorder.py noise_off --secs 4 --link 'M1>M3' --label tx_off       # PN1, no TX
python3 siggen.py zeros --secs 40                                              # FN1
python3 iq_recorder.py noise_zeros --secs 4 --link 'M1>M3' --label tx_zeros   # PN1
# Scripts 3, 4, 6: bursts in four channels
python3 siggen.py bursts --rms-dbfs -24 --secs 40                              # FN1
python3 iq_recorder.py bursts --secs 5 --link 'M1>M3' --label bursts           # PN1
python3 occupancy_monitor.py bursts --noise noise_zeros --block-ms 0.5 --plot \
        --channels=-750e3:250e3,-250e3:250e3,250e3:250e3,750e3:250e3
python3 fixed_tune_scanner.py bursts --noise noise_zeros --dwell-ms 0.5 --plot \
        --channels=-750e3:200e3,-250e3:200e3,250e3:200e3,750e3:200e3
# Scripts 2, 10: tone with CHEM frequency-offset steps during a 30 s record
python3 siggen.py tone --f0 100e3 --secs 70                                    # FN1
python3 iq_recorder.py tone_fo --secs 30 --link 'M1>M3' --label tone_fo        # PN1
CC fo 300 'M1>M3'; sleep 10; CC fo -200 'M1>M3'                                # during the record
python3 doppler_cfo.py tone_fo --f0 100e3 --plot
# Scripts 7, 11: SNR sweeps (a reference is one period, written without radio)
python3 siggen.py qpsk --save qpsk_ref.c64 --rms-dbfs -20
python3 siggen.py 16qam --save qam16_ref.c64 --rms-dbfs -20
python3 siggen.py qpsk --rms-dbfs -20 --secs 250                               # FN1
python3 chem_sweeper.py noise 30,25,20,15,10,5,0,LINK_BUDGET --link 'M1>M3' \
        --ref qpsk_ref.c64 --mod qpsk --restore LINK_BUDGET --out sweep_qpsk   # PN1
python3 meters.py sweep_16qam_noise_LINK_BUDGET --ref qam16_ref.c64 --mod 16qam \
        --sig-bw 675e3 --plot meters.png
# Script 5: sounder with CHEM CIR changes while the RX stream runs
python3 channel_sounder.py tx --seq zc --secs 110                              # FN1
python3 chem_sweeper.py cir clear,three,dense8,echo2 --link 'M1>M3' --settle 9 \
        --secs 2 --save-iq --restore clear --out snd                           # PN1
python3 channel_sounder.py ana snd_cir_three --seq zc --plot
# Script 12: watchdog during a reset (on FN1)
python3 chem_watchdog.py --grace 5 --period 2 --secs 180 --max-resets 1 --out watchdog
# Script 14: copy of the UHD2 sample with in-band LOs, then the conversion (on PN1)
cp -r /root/AERPAW-Dev/AHN/E-VM/Profile_software/SDR_control/LTE_IQ uhd2; cd uhd2
sed -i 's|^fLO = .*|fLO = (3400e6, 3410e6)|' myConstants.py
sed -i 's|^total_duration = .*|total_duration = 60|' myConstants.py
python3 get_IQ.py
python3 ../mat2sigmf.py mat '/root/Results/results_34*.mat' --out-dir uhd2_sigmf
```

10.4. Commands for the cell tests

The LTE cell uses the SE1 sample scripts of the image (`startSRSRAN-SISO-EPCandENB.sh` on FN1, `startSRSRAN-SISO-UE.sh` on PN1). The 5G cell uses the commands of Report-2: OCUDU gNB and Open5GS on FN1, OAI NR UE on PN1, band n78, 3404.64 MHz, 10 MHz, 15.36 MS/s.

```
# LTE: monitor node PN2
python3 dl_capture.py capture lte_idle --rat lte --rate 11.52e6 --freq 3410e6 --secs 2
iperf3 -c <UE_IP> -t 30 -i 1                                                   # FN1
python3 iq_recorder.py lte_load --rate 11.52e6 --freq 3410e6 --secs 2          # PN2
python3 occupancy_monitor.py lte_load --rate 11.52e6 --nchan 10 --span 9e6 --thr-db 20 --plot
```

```
# 5G: monitor node PN2, logger and metrics on FN1
python3 dl_capture.py capture nr_dl --rat nr --rate 15.36e6 --freq 3404.64e6 --secs 0.6
python3 dl_capture.py nrsib1 nr_dl --decoder <path>/nrsib1_decode
python3 link_logger.py --target <UE_IP> --secs 45 --iperf-secs 25 --iperf-delay 5 \
        --link 'M1>M3' --plot
python3 ws_tool.py metrics ws_metrics.jsonl               # gNB JSON metrics, port 8001
```

Remove a stale `srs_spgw_sgi` interface on FN1 before a 5G session, if no srsEPC runs. It holds the address 172.16.0.1 and can break the DL route to the 5G UE.

10.5. Commands for the new experiments of 5 November 2026

The commands of Section 7 are in `sess_r6.sh` and `sess_r6b.sh`. The helper `r6_ana.py` runs on the node. It uses only the CPU, not the radio.

```
python3 chem_sweeper.py noise 30,0,20,5,25,10,15 --link 'M1>M3' --settle 3 --secs 2 \
        --ref qpsk_ref.c64 --mod qpsk --restore LINK_BUDGET --save-iq --out ds_qpsk   # R1
python3 ds_manifest.py ds_qpsk.csv ds_16qam.csv --annotate --out manifest.csv   # R1, offline
python3 r6_ana.py step snr_step --block-ms 10            # R1, R3: power steps in time
python3 iq_recorder.py nr_25M --rate 15.36e6 --freq 3404.64e6 --secs 2 --link 'M1>M4'  # R2
python3 r6_ana.py fold nr_25M --noise nr_noise           # R2: duty cycle and TDD fold
python3 r6_ana.py psd sg_16qam --nfft 4096               # R3: spectra of the Script 6 figure
```

10.6. Commands for the new experiments of 10 November 2026

The commands of Section 7.4 to 7.7 are in `sess_r5py.sh`. All runs use the session frame above.

```
python3 siggen.py tone --freq 914e6 --gain 45 --secs 45                  # N1, FN1
python3 preflight.py rx --freq 914e6 --secs 5 --out pf_914               # N1, PN1
python3 chem_sweeper.py noise 20,10,LINK_BUDGET --freq 914e6 --link 'M1>M3' --ref qpsk_ref.c64 \
        --mod qpsk --settle 3 --secs 2 --restore LINK_BUDGET --out sw914  # N1, PN1, TX: siggen qpsk
python3 iq_recorder.py lag2M --secs 120 --lag-log --link 'M1>M3'         # N2, PN1
python3 r6_ana.py lag lag2M                                              # N2: steps above 0.4 ms
python3 siggen.py bursts --chan-offsets 0 --chan-bw 1.5e6 --duty 0.25 --period-ms 10 \
        --rms-dbfs -24 --secs 90                                         # N4, FN1
python3 chem_sweeper.py noise 20,10,LINK_BUDGET --link 'M1>M3' --settle 3 --secs 4 \
        --save-iq --restore LINK_BUDGET --out bsnr                       # N4, PN1
python3 r6_ana.py bsnr bsnr_noise_20                                     # N4: SNR in the bursts
python3 siggen.py zeros --rate 7.68e6 --secs 40                          # N3, FN1
python3 iq_recorder.py z_7.68e6 --rate 7.68e6 --secs 4 --link 'M1>M3'   # N3, PN1
```

References

[1] I. Guvenc, "Report-1: GNU Radio Experiments on GitHub with Toy AERPAW Deployments," NC State University, Nov. 2026.

[2] I. Guvenc, "Report-2: OCUDU 5G on AERPAW: 5G Standalone with the OAI NR UE in the AERPAW Digital Twin," NC State University, Nov. 2026.

[3] I. Guvenc, "Report-2 Part-2: OCUDU UAV Flights and Link Drops," NC State University, Nov. 2026.

[4] I. Guvenc, "Report-3: OTFS and OFDM in the AERPAW Digital Twin: GitHub Deployment, Receivers and Drone Flights," NC State University, Nov. 2026.

[5] I. Guvenc, "Report-4: OAI 5G on AERPAW," NC State University, Nov. 2026.

[6] I. Guvenc, "Report-6: AERPAW AORA Challenge: Research Concepts, Repositories and Digital-Twin Results," NC State University, Nov. 2026.

[7] I. Guvenc, "AERPAW Digital Twin Debug Tests: Experiment X1532," NC State University, Nov. 2026.

[8] AERPAW, "AERPAW User Manual." [Online]. Available: https://sites.google.com/ncsu.edu/aerpaw-user-manual/

[9] Ettus Research, "USRP Hardware Driver and USRP Manual, version 4.9." [Online]. Available: https://files.ettus.com/manual/

[10] GNU Radio SigMF project, "The Signal Metadata Format (SigMF) specification." [Online]. Available: https://github.com/sigmf/SigMF

Appendix A. Glossary

Table 20: Terms and abbreviations of this report.

Term	Meaning
ADS-B	Automatic dependent surveillance-broadcast.
AERPAW	Aerial Experimentation and Research Platform for Advanced Wireless.
AI	Artificial intelligence.
Annotation (SigMF)	A label for a range of samples in the `.sigmf-meta` file.
AORA	AERPAW Open Research on Autonomy.
API	Application programming interface.
AWGN	Additive white Gaussian noise.
BER	Bit error rate.
CBRS	Citizens broadband radio service.
CCDF	Complementary cumulative distribution function. For PAPR: the share of samples above a level.
CFAR	Constant false alarm rate.
CFO	Carrier frequency offset: the difference between the TX and RX carrier frequencies.
CHEM	Channel emulator of the AERPAW digital twin. It applies the channel of each link to the samples.
CIR	Channel impulse response. In CHEM, a list of multipath taps with delays in samples.
CNN	Convolutional neural network.
CORESET	Control resource set: the time-frequency region of the NR control channel (PDCCH).
CPU	Central processing unit.
C-VM, E-VM	Control virtual machine of the twin, experiment virtual machine (your node).
dBFS	Decibels relative to full scale. A sample with magnitude 1 is at 0 dBFS.
DCI	Downlink control information, sent on the PDCCH.
DL, UL	Downlink (base station to UE), uplink (UE to base station).
DSP	Digital signal processing.
Duty cycle	The share of time blocks in which a channel is occupied (energy above the threshold).
E2, KPM	O-RAN interface between the gNB and the near-RT RIC. KPM: key performance measurements over E2.
eNB, gNB	LTE base station, 5G NR base station.
EVM	Error vector magnitude: the rms error of the received symbols, in percent of the rms symbol.
`fc32`, `sc16`, `cf32_le`	Sample formats: complex 32-bit float (host), complex 16-bit integer (wire), SigMF name of `fc32`.
FFT	Fast Fourier transform.
FN, PN	Fixed node, portable node.
FPGA	Field-programmable gate array.
GNSS	Global navigation satellite system.
GPSDO	GPS-disciplined oscillator.
GPU	Graphics processing unit.
GUI	Graphical user interface.
HARQ	Hybrid automatic repeat request: retransmission with combining in LTE and NR.
ID	Identifier.

Term	Meaning
IP	Internet protocol.
I/Q	In-phase and quadrature parts of a complex baseband sample.
ISAC	Integrated sensing and communication.
LDPC	Low-density parity-check.
LO	Local oscillator. Its frequency is the center frequency of the record.
LTE	Long-term evolution.
LW1, LW2	The two fixed AERPAW sites of the twin nodes FN1 and FN2.
MAD	Median absolute deviation.
MCS	Modulation and coding scheme.
MIB, SIB1	Master information block, system information block 1: the first broadcast messages of an NR cell.
ML	Machine learning.
n78	5G NR band 3300 to 3800 MHz.
near-RT RIC	Near-real-time RAN intelligent controller.
NID2	The part (0, 1 or 2) of the cell identity that the PSS carries.
NR	New radio.
OBW	Occupied bandwidth.
OCUDU, OAI	The 5G gNB software of Report-2, and OpenAirInterface (5G gNB and UE software).
OEO	AERPAW operator interface that controls the vehicles of an experiment.
OFDM	Orthogonal frequency-division multiplexing.
OTFS	Orthogonal time frequency space.
PAPR	Peak-to-average power ratio.
PBCH	Physical broadcast channel.
PCI	Physical cell identity.
PDCCH, PDSCH, PUSCH	NR physical downlink control channel, physical downlink shared channel, physical uplink shared channel.
PDP	Power delay profile: the power of the channel taps against the delay.
PHY	Physical layer.
PLMN	Public land mobile network.
PRACH	Physical random-access channel.
PRB	Physical resource block: 12 subcarriers (360 kHz at 30 kHz spacing).
PSD	Power spectral density.
PSK	Phase-shift keying.
PSS, SSS, SSB	Primary and secondary synchronization signals, and the synchronization signal block that holds them.
QAM	Quadrature amplitude modulation.
QPSK	Quadrature phase-shift keying.
RAN	Radio access network.
RIC	RAN intelligent controller.
RLF	Radio link failure.
RMS	Root mean square.
RRC (filter)	Root-raised-cosine pulse shape of the QPSK and 16QAM test signals. It is not the 5G radio resource control protocol.
RSSI	Received signal strength indicator.
RTT	Round-trip time (ping).
SDR	Software-defined radio.

Term	Meaning
SE1	The srsRAN LTE sample experiment of the AERPAW image.
SFN	System frame number (0 to 1023, one frame is 10 ms).
SIB	System information block.
SigMF	Signal Metadata Format: raw samples plus JSON metadata.
SNR	Signal-to-noise ratio. CHEM sets it over the full sample band.
SSH	Secure shell.
STFT	Short-time Fourier transform.
TAC	Tracking area code.
TCP	Transmission control protocol.
TDD	Time-division duplex: DL and UL use the same frequency in different slots.
UAV	Unmanned aerial vehicle.
UDP	User datagram protocol.
UE	User equipment (the phone or modem side).
UHD	USRP Hardware Driver of Ettus Research, with a C++ and a Python API.
UHD1, UHD2	The AERPAW Python sample experiments for spectrum monitoring and for I/Q collection.
USRP	Universal software radio peripheral.
VPN	Virtual private network.
V-USRP	Virtual USRP of the digital twin. It sends and receives samples through CHEM.
WebSocket	A two-way connection over TCP. The OCUDU gNB gives its JSON metrics on port 8001.
ZC	Zadoff-Chu.

Appendix B. Script Index and File Locations

Table 21 gives the file, the result folder and the log files of each script. The scripts are in `node_tools/aerpaw_py/` in our working folder and in `/root/claude_tools/aerpaw_py/` on the nodes. Each result folder in `gnuradio_deploy/results/` has `result.json`, `fig.png` and the folder `logs/`. The column "Research name" gives the name of the script in the research report (`reports/Python SDR scripts for AERPAW.md`).

Table 21: Script index and file locations.

#	File	Lines	Research name	Result folder and logs
1	`vusrp_io.py`	365	shared helper	`pyb_01`: `S1_M3_noise_off.log`, `S1_M3_noise_zeros.log`, `S1_run_timeline.txt`, `off_offline_test.json`
2	`iq_recorder.py`	68	`sigmf_rec.py`	`pyb_02`: `S2_M3_tone_fo.sigmf-meta`, `S2_M3_tone_fo_chem.csv`
3	`occupancy_monitor.py`	106	`occupancy_mon.py`	`pyb_03`: `S1_M3_occ_bursts.json`, `S3b_M4_occ_lte_*.json`, `S4b_M4_occ_nr_*.json`
4	`fixed_tune_scanner.py`	76	`fixed_tune_scan.py`	`pyb_04`: `S1_M3_scan_bursts.json`, `S1_M3_scan_bursts_d05.json`
5	`channel_sounder.py`	73	`sounder.py`	`pyb_05`: `S2_M3_snd_cir_*_pdp.json`, `S2_M3_snd.json`, `S1_M3_snd_three_pdp.json`
6	`siggen.py`	156	`siggen.py`	`pyb_06`: `S1_M1_tx_*.log`, `S2_M1_tx_*.log`
7	`meters.py`	105	`meters.py`	`pyb_07`: `S2_M3_sweep_qpsk.json`, `S2_M3_sweep_16qam.json`, `off_offline_test.json`
8	`dl_capture.py`	120	`dl_capture_decode.py`	`pyb_08`: `S3b_M4_lte_idle_pss.json`, `S4_M4_nr_dl_nrsib1.json`, `S4b_M4_nr_dl_pss.json`
9	`link_logger.py`	123	`linkq_logger.py`	`pyb_09`: `S4b_M1_linklog.json`, `S4b_M1_linklog_1s.csv`, `S4b_M1_linklog_iperf.txt`, first run `S4_M1_linklog.json`
10	`doppler_cfo.py`	82	`doppler_cfo_est.py`	`pyb_10`: `S2_M3_tone_fo_cfo.json`, `S2_M3_tone_fo_chem.csv`
11	`chem_sweeper.py`	78	`chem_sweep.py`	`pyb_11`: `S2_M3_sweep_qpsk.csv`, `S2_M3_sweep_16qam.csv`, `S2_M3_snd.csv`
12	`chem_watchdog.py`	72	`chem_watchdog.py`	`pyb_12`: `S1_M1_watchdog.*`, `S2_M1_watchdog.*`
13	`preflight.py`	87	`vusrp_selftest.py`	`pyb_13`: `S1_M1_check_m1.log`, `S1_M3_check_m3.log`, `S1_M3_pf_rx.log`
14	`mat2sigmf.py`	80	`mat2sigmf.py`	`pyb_14`: `S2_M3_uhd2_getIQ.log`, `S2_M3_uhd2_sigmf_*.sigmf-meta`
–	`ds_manifest.py`	50	– (new, 5 Nov)	`pyb_15`: `manifest.csv`, annotated `ds_*.sigmf-meta`
–	`r6_ana.py`	133	– (new, 5 Nov)	`pyb_15`: R1 data set and fixed-SNR step. `pyb_16`: R2 OCUDU cell with gNB metrics. `pyb_17`: R3 command timing and spectra of Fig. 12. `pyb_18` to `pyb_21`: N1 to N4 (10 Nov)

Other files. This report is in `report_python/`: `main.tex`, the section files (`sec_*.tex`, `app_glossary.tex`, `claude_section.tex`), `figures/` and `figures_claude/`. The scripts in `tools/` draw the figures: `make_figs.py`, `data_figs.py`, `new_exp.py`, `concept_figs.py` and `nodes_fig.py`. The session scripts are in `gnuradio_deploy/work/deployX/` and `results/R5py_nov10/`. The research report is `reports/Python SDR scripts for AERPAW.md`.

Appendix C. Partial and Failed Items

All 14 scripts passed their tests in the twin. Some parts were not tested, and some first attempts failed. Table 22 gives these items.

Table 22: Parts that we did not test, and first attempts that failed.

Script	Item
5 `channel_sounder`	First attempt failed (session S1). The CIR commands came after the RX had stopped. CHEM applied them to no link, and the PDP showed one tap. The second attempt (S2) with the RX stream open passed.
9 `link_logger`	First run failed in part (session S4). The parser did not read the intervals with 0 bits/sec, and it reported 3.68 Mbit/s over 10 intervals. We corrected the parser and repeated the test (S4b).
6 `siggen`	The kinds multitone, chirp, pn and noise were saved offline only, not sent on air. The option `--gain-sched` was not tested on air.
10 `doppler_cfo`	Only the CFO path was tested. The drone was on the ground, so the CHEM Doppler was 0 Hz. A flight test is open.
12 `chem_watchdog`	The action path ran only during a planned reset. No unplanned CHEM stop occurred.
4 `fixed_tune_scanner`	Tested offline on a live record. The `--live` mode was not used.
3 `occupancy_monitor`	The LTE and NR floors are estimates (10th percentile of the median PSD), because no TX-off record was made at 11.52 and 15.36 MS/s. The NR load duty includes the UE uplink.
8 `dl_capture`	10 of 26 windows had no SIB1, because the Type0-PDCCH occasion was outside the window. The LTE path has only the PSS check, no LTE decoder.
UHD1 sample	Not run, because it stops CHEM for all users.
Research script 14	The optional bistatic FMCW or ZC range-Doppler script of the research report was not written.
Public repositories	None of the repositories of Section 8 was run in the twin.
R1 (5 Nov)	The 3-s settle time was tested for fixed-SNR sweeps only. The large `.sigmf-data` files were deleted on the node after the metrics. The meta files and the manifest are in `pyb_15`.
R2 (5 Nov)	One run, 2-s records. The monitor fold and the gNB slot index are not aligned (no slot timing at the monitor).
R3 (5 Nov)	The last command (frequency offset 0) came after the end of the record and has no measurement.